



MONASH
University

MONASH
INFORMATION
TECHNOLOGY

Week 3

The Relational Database Model

Workshop 2022 Semester 1



Overview

Once we have a conceptual model, it is time to move to the second stage and map this to a logical model

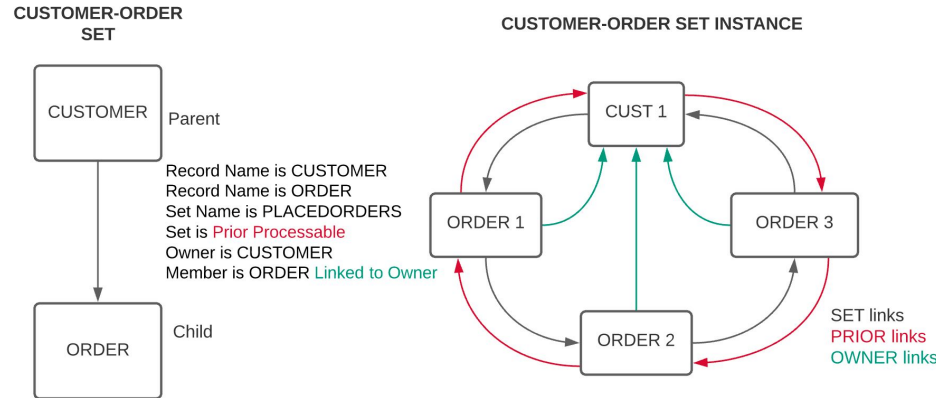
For our unit this will involve mapping to the *Relational Model* in preparation for implementation in a RDBMS. First before we consider this mapping it is necessary to have a clear understanding of the Relation Model and its use:

- Relational Model
- Relational Algebra

Early Database Models

- Hierarchical (1970's eg. IBM Information Management System (IMS))
 - 1:M relationships, a tree of linked records, child has only one parent
- Network (1970's eg. Integrated Data Store IDS, basis for the CODASYL group)
 - child may have multiple parents
- Both Navigational - move around in data via *embedded links* (pointers)

Network:



The Relational Model

- Introduced by CODD in 1970 - the fundamental basis for the relational DBMS
- Basic structure is the mathematical concept of a RELATION mapped to the 'concept' of a table (tabular representation of relation)
 - Relation - abstract object
 - Table - pictorial representation
 - Storage structure - "real thing" - eg. isam file of 1's and 0's
- Relational Model Terminology
 - DOMAIN - set of atomic (indivisible) values
 - Examples (name, data type, data format):
 - customer_number domain - 5 character string of the form xxxdd
 - name domain - 20 character string
 - address domain - 30 character string containing street, town & postcode
 - credit_limit domain - money in the range \$1,000 to \$99,999

A Relation

- A relation consists of two parts
 - heading
 - body
- **Relation Heading**
 - Also called Relational **Schema** consists of a fixed set of attributes
 - $R(A_1, A_2, \dots, A_n)$
 - R = relation name, A_i = attribute i
 - Each attribute corresponds to one underlying domain:
 - Customer relation heading:
 - CUSTOMER (custno, custname, custadd, custcredlimit)
 - » $\text{dom}(\text{custno}) = \text{customer_number}$
 - » $\text{dom}(\text{custname}) = \text{name}$
 - » $\text{dom}(\text{custadd}) = \text{address}$
 - » $\text{dom}(\text{custcredlimit}) = \text{credit_limit}$

custno	custname	custadd	custcredlimit
--------	----------	---------	---------------

Relation Body

▪ Relation Body

- Also called Relation Instance (state of the relation at any point in time)
 - $r(R) = \{t_1, t_2, t_3, \dots, t_m\}$
 - consists of a time-varying set of n-tuples
 - Relation R consists of tuples $t_1, t_2, t_3 \dots t_m$
 - m = number of tuples = **relation cardinality**
 - each n-tuple is an ordered list of n values
 - $t = \langle v_1, v_2, \dots, v_n \rangle$
 - n = number of values in tuple (no of attributes) = **relation degree**
- In the tabular representation:
 - Relation heading $\Rightarrow$ column headings
 - Relation body $\Rightarrow$ set of data rows

custno	custname	custadd	custcredlimit
SMI13	SMITH	Wide Rd, Clayton, 3168	2000
JON44	JONES	Narrow St, Clayton, 3168	10000
BRO23	BROWN	Here Rd, Clayton, 3168	10000

Relation Properties

- **No duplicate tuples**
 - by definition sets do not contain duplicate elements
 - hence tuples must be unique
- **Tuples are unordered within a relation**
 - by definition sets are not ordered
 - hence tuples can only be accessed by content
- **No ordering of attributes within a tuple**
 - by definition sets are not ordered

Relation Properties cont'd

- **Tuple values are atomic** - cannot be divided
 - EMPLOYEE (eid, ename, departno, dependants)
 - not allowed: dependants (dep_name, dep_age) multivalued
 - hence no multivalued (repeating) attributes allowed, called the first normal form rule
- COMPARE with tabular representation
 - normally nothing to prevent duplicate rows
 - rows are ordered
 - columns are ordered
 - tables and relations are not the same 'thing'

Functional Dependency

■ Functional Dependency:

- A set of attributes A functionally determines an attribute B if, and only if, for each A value, there is exactly one value of B in the relation. It is denoted as $A \rightarrow B$ (A determines B, or B depends on A)
 - $orderno \rightarrow orderdate$
 - $prodno \rightarrow proddesc$
 - $orderno, prodno \rightarrow qtyordered$

ORDERNO	ORDERDATE
10	01/MAY/19
11	02/MAY/19
12	03/MAY/19
13	04/MAY/19
14	04/MAY/19
15	05/MAY/19
16	06/MAY/19

ORDERNO	PRODNO	QTYORDERED	LINEPRICE
10	101	1	11.98
11	101	1	11.98
11	103	2	123.58
12	104	10	479.8
13	105	2	140.36
14	106	1	31.99
15	107	3	116.73

PRODNO	PRODDISC	PRODUNITPRICE
101	Salmon - Smoked, Sliced	11.98
102	Brocolinni - Gaylan, Chinese	80.75
103	Pasta - Lasagne, Fresh	61.79
104	Melon - Cantaloupe	47.98
105	Wine - Peller Estates Late	70.18
106	Peas - Pigeon, Dry	31.99
107	Pumpkin - Seed	38.91

Relational Model Keys

- A **superkey** of a relation R is an attribute or set of attributes which exhibits only the uniqueness property
 - No two tuples of R have the same value for the superkey (Uniqueness property)
 - $t1[\text{superkey}] \neq t2[\text{superkey}]$
- A **candidate key (CK)** of a relation R is an attribute or set of attributes which exhibits the following properties:
 - Uniqueness property (as above), *and*
 - No proper subset of CK has the uniqueness property (Minimality or Irreducibility property) ie. a minimal superkey
- One candidate key is chosen to be the **primary key (PK)** of a relation. Remaining candidate keys are termed alternate keys (AK).

Many possible superkeys

Potentially many possible candidate keys

Only ONE primary key (may be composed of many attributes - a composite primary key)

Free text answer in Flux (after Group discussion):

Q1. List all the super keys for:

Unit_code	Mark	Studid	Year
FIT2094	80	111	2016
FIT2094	20	111	2015
FIT2004	100	111	2016
FIT2004	40	222	2015
FIT2004	40	333	2015

Treat this as only a small sample of the data

Selection of a Primary key

- **A primary key must be chosen considering the data that *may be added to the table in the future***
 - Names, dates of birth etc are rarely unique and as such are not a good option
 - PK should be free of 'extra' semantic meaning and security compliant, preferably a single attribute, preferably numeric (see Table 5.3 Coronel & Morris)
 - Natural vs Surrogate primary key
 - ENROLMENT (unitcode, student_id, enrol_sem, enrol_year, enrol_mark, enrol_grade)
 - Superkey
 - CK
 - PK
 - Issues with PK?

TABLE 5.3

DESIRABLE PRIMARY KEY CHARACTERISTICS

PK CHARACTERISTIC	RATIONALE
Unique values	The PK must uniquely identify each entity instance. A primary key must be able to guarantee unique values. It cannot contain nulls.
Nonintelligent	The PK should not have embedded semantic meaning other than to uniquely identify each entity instance. An attribute with embedded semantic meaning is probably better used as a descriptive characteristic of the entity than as an identifier. For example, a student ID of 650973 would be preferred over Smith, Martha L. as a primary key identifier.
No change over time	If an attribute has semantic meaning, it might be subject to updates, which is why names do not make good primary keys. If Vickie Smith is the primary key, what happens if she changes her name when she gets married? If a primary key is subject to change, the foreign key values must be updated, thus adding to the database work load. Furthermore, changing a primary key value means that you are basically changing the identity of an entity. In short, the PK should be permanent and unchangeable.
Preferably single-attribute	A primary key should have the minimum number of attributes possible (irreducible). Single-attribute primary keys are desirable but not required. Single-attribute primary keys simplify the implementation of foreign keys. Having multiple-attribute primary keys can cause primary keys of related entities to grow through the possible addition of many attributes, thus adding to the database workload and making (application) coding more cumbersome.
Preferably numeric	Unique values can be better managed when they are numeric, because the database can use internal routines to implement a counter-style attribute that automatically increments values with the addition of each new row. In fact, most database systems include the ability to use special constructs, such as Autonumber in Microsoft Access, sequence in Oracle, or uniqueidentifier in MS SQL Server to support self-incrementing primary key attributes.
Security-compliant	The selected primary key must not be composed of any attribute(s) that might be considered a security risk or violation. For example, using a Social Security number as a PK in an EMPLOYEE table is not a good idea.

Null in the Relational Model *Implementation*

- ***NULL is a concept created and implemented by SQL, does not exist in classical relational algebra***
- NULL is NOT a value - is a representation of the fact that there is *NO VALUE*
- Reasons for a NULL:
 - VALUE NOT APPLICABLE -
 - EMP relation - empno, deptno, salary, commission
 - commission only applies to staff in sales dept
 - VALUE UNKNOWN -
 - Joe's salary is NULL, Joe's salary is currently unknown
 - VALUE DOES NOT EXIST -
 - Tax File Number - is applicable to all employees but Joe may not have a number at this time
 - VALUE UNDEFINED -
 - Certain items explicitly undefined eg. divide by zero
 - Columns Number_of_payments, Total_payments
 - Column Average_payment_made
 - If Number_of_payments = 0 => Average undefined

Writing Relations

- Relations may be represented using the following notation:
 - `RELATION_NAME (attribute1, attribute2,...)`
- Relation_name must not be pluralised (is a set name)
- The primary key is underlined.
- Example:
 - `STAFF (staff_id, staff_surname, staff_initials, staff_address, staff_phone)`

Q2. A well designed relational database (a database based on the relational model) has:

- A. No redundant data
- B. Minimal redundant data
- C. A large amount of redundant data
- D. A level of redundancy based on the vendors implementation

Relational Database

- A relational database is a collection of normalised relations.
- Normalisation is part of the design phase of the database and will be discussed next week.

Example relational database:

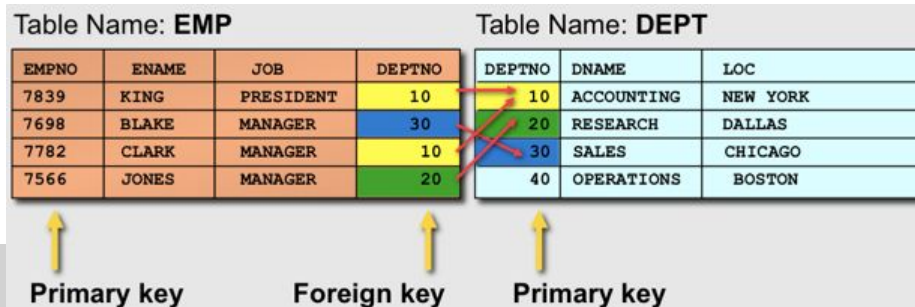
ORDER (order_id, order_date)

ORDER_LINE (order_id, prod_id, ol_quantity)

PRODUCT (prod_id, prod_desc, prod_uprice)

Foreign Key (FK) - Implementation

- FK: An attribute/s in a relation that exists in the same, or another relation as a Primary Key.
- Referential Integrity
 - A Foreign Key value must either *match* the **full primary key** in a relation **or be NULL**.
- The pairing of PK and FK creates relationships (**logical** connections) between tables when implemented in a RDBMS. Hence the abstraction away from the underlying storage model.



Q3. Business rules:

Runners may form a team, the runner who registers the team is recorded as the team leader. Each team can have up to 5 members (runners).

Identify the FK(s):

TEAM(team_id, team_name, team_leader)

RUNNER(runner_id, runner_name, team_id)

- A. team_leader in TEAM
- B. team_id in TEAM
- C. runner_id in RUNNER
- D. team_id in RUNNER

Data Integrity - Implementation

- Entity integrity
 - Primary key value must not be NULL.
 - No duplicate tuple property then ensures that each primary key must be unique
 - Implemented in the RDBMS via a unique index on the PK
- Referential integrity
 - The values of FK must either match a value of a full PK in the related relation or be NULL.
- Column/Domain integrity
 - All values in a given column must come from the same domain (the same data type and range).

Q4. The following set of relations:

HOSPITAL (hosp_id, hosp_name, hosp_phone)

DOCTOR (hosp_id, dr_id, dr_name, dr_mobile)

PATIENT (pat_id, pat_name, pat_dob, dr_id)

- A. have no integrity issues
- B. violate entity integrity
- C. violate referential integrity
- D. violate column/domain integrity

Multiple responses allowed

Relational DMLs

- Relational Calculus
- Relational Algebra
- Transform Oriented Languages (e.g. SQL)
- Graphical Languages
- Exhibit the “closure” property - queries on relations produce relations

Relational Calculus

- Based on mathematical logic.
- Non-procedural.
- Primarily of theoretical importance.
- May be used as a yardstick for measuring the power of other relational languages (“relational completeness”).
- Operators may be applied to any number of relations.

RELATIONAL ALGEBRA

Manipulation of relational data

Relational Algebra

- Relationally complete.
- Procedural.
- Operators only apply to at most two relations at a time.
- 8 basic operations:
 - *single relation*: selection, projection
 - *two relations*:
 - cartesian product, join
 - union
 - intersection
 - difference
 - division
- *Standard/pure RA form has no concept of NULL (Database unit uses standard RA)*

Relational Operation PROJECT

PRDETAIL (project_code, project_manager, project_bid_price)

π

PROJECT_CODE	PROJECT_MANAGER	PROJECT_BID_PRICE
21-5Z	Holly B. Parker	\$16,833,460.00
25-2D	Jane D. Grant	\$12,500,000.00
25-5A	George F. Dorts	\$32,512,420.00
25-9T	Holly B. Parker	\$21,563,234.00
27-4Q	George F. Dorts	\$10,314,545.00
29-2D	Holly B. Parker	\$25,559,999.00
31-7P	William K. Moor	\$56,850,000.00

RESULT = $\pi_{\text{project_manager}}$ PRDETAIL

Relational Operation SELECT

PRDETAIL (project_code, project_manager, project_bid_price)

σ

PROJECT_CODE	PROJECT_MANAGER	PROJECT_BID_PRICE
21-5Z	Holly B. Parker	\$16,833,460.00
25-2D	Jane D. Grant	\$12,500,000.00
25-5A	George F. Dorts	\$32,512,420.00
25-9T	Holly B. Parker	\$21,563,234.00
27-4Q	George F. Dorts	\$10,314,545.00
29-2D	Holly B. Parker	\$25,559,999.00
31-7P	William K. Moor	\$56,850,000.00

RESULT = $\sigma_{\text{project_code} = 25-5A}$ PRDETAIL

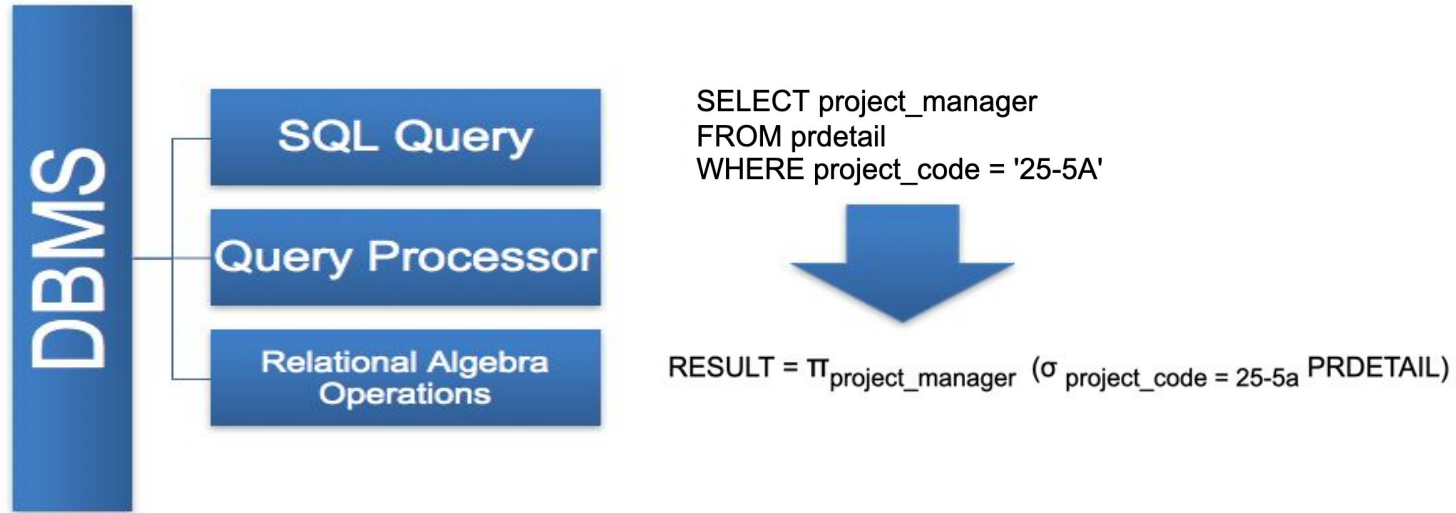
Relational Operation Multiple Actions

PRDETAIL (project_code, project_manager, project_bid_price)

	2	
	PROJECT_CODE	PROJECT_MANAGER
	PROJECT_BID_PRICE	
	21-5Z	Holly B. Parker
	25-2D	Jane D. Grant
1	25-5A	George F. Dorts
	25-9T	Holly B. Parker
	27-4Q	George F. Dorts
	29-2D	Holly B. Parker
	31-7P	William K. Moor

RESULT = $\pi_{\text{project_manager}} (\sigma_{\text{project_code} = 25-5A} \text{PRDETAIL})$

SQL vs Relational Algebra in the Database



Q5. Relational Algebra *select* and *project*

The following relations represent a karate dojo member training attendance:

SENSEI (sensei_id, sensei_name)

TRAINING_SCHEDULE (training_day, training_time, group_id, sensei_id)

ATTENDANCE (training_day, training_time, member_id, attendance_date)

MEMBER (member_id, member_name, member_dob, member_belt, group_id)

GROUP (group_id, group_name, group_age_range)

- A. Primary keys are underlined
- B. A karate member falls into one of the age level groups: Tiny Tiger (for 4-7 year old), Young Dragon (for 8-14 years old), or Adult (for 14+ years old) and owns a certain color of belt (e.g. white, green, brown or black)
- C. Sensei (Karate teachers) are scheduled to train an age level group of karate members in a particular day and time (e.g. Sensei Luke Nakamura trains Tiny Tiger members every Tuesday 5pm)
- D. A karate member may attend more than one training schedule of their age level group in a given week.

Write the relational algebra for the following query (**your answer must show an understanding of query efficiency**):

(1) Show the name and dob of all black belt members.

JOIN

- Join operator used to combine data from two or more relations, based on a common attribute or attributes.
- For conventional relational algebra, different types of joins are:
 - theta-join
 - equi-join
 - natural join

THETA JOIN (Generalised join)

$$(\text{Relation_1}) \bowtie_F (\text{Relation_2})$$

- F is a predicate (i.e. truth-valued function) which is of the form $\text{Relation_1.a}_i \theta \text{Relation_2.b}_i$
 - $\text{CUSTOMER.cust_no} \theta \text{ORDER.cust_no}$
- θ is one of the standard arithmetic comparison operators,
 $<, \leq, =, \geq, >$
- Most commonly, θ is equals ($=$), but can be *any* of the operators
 - $\text{EMPLOYEE.emp_sal} > \text{SALARYSCALE.step_5}$

NATURAL JOIN

STUDENT		MARK		
ID	Name	ID	Subj	Marks
1	Alice	1	1004	95
2	Bob	2	1045	55
		1	1045	90

Step 1: STUDENT X MARK

Step 2: delete rows where IDs do not match (select =)

Result at Step 2 is an Equijoin

STUDENT.ID	Name	MARK.ID	Subj	Marks
1	Alice	1	1004	95
1	Alice	2	1045	55
1	Alice	1	1045	90
2	Bob	1	1004	95
2	Bob	2	1045	55
2	Bob	1	1045	90

NATURAL JOIN

STUDENT		MARK		
ID	Name	ID	Subj	Marks
1	Alice	1	1004	95
2	Bob	2	1045	55
		1	1045	90

Step 1: STUDENT X MARK

Step 2: delete rows where IDs do not match (select =)

Step 3: delete duplicate columns (project away)

Result at Step 3 is a Natural Join

STUDENT.ID	Name	MARK.ID	Subj	Marks
1	Alice	1	1004	95
1	Alice	1	1045	90
2	Bob	2	1045	55

NATURAL JOIN

STUDENT			MARK		
ID	Name		ID	Subj	Marks
1	Alice	⋈	1	1004	95
2	Bob		2	1045	55
			1	1045	90

Step 1: STUDENT X MARK

Step 2: delete rows where IDs do not match (select =)

Step 3: delete duplicate columns (project away)

ID	Name	Subj	Marks
1	Alice	1004	95
1	Alice	1045	90
2	Bob	1045	55

A natural join of STUDENT and MARK

Q6. Which of the following statements returns a natural join of the two relations on the agent code (agent_cd and agent_code)?

Relation : CUSTOMER

CUS_CODE	CUS_LNAME	CUS_ZIP	AGENT_CD
1132445	Walker	32145	231
1217782	Adares	32145	125
1312243	Rakowski	34129	167
1321242	Rodriguez	37134	125
1542311	Smithson	37134	333
1657399	Varloo	32145	231

Relation : AGENT

AGENT_CODE	AGENT_PHONE
125	6152439887
167	6153426778
231	6152431124
333	9041234445

- A. $\sigma_{\text{agent_cd} = \text{agent_code}} (\text{CUSTOMER X AGENT})$
- B. $\pi_{\text{cus_code, cus_lname, cus_zip, agent_code, agent_phone}} (\sigma_{\text{agent_cd} = \text{agent_code}} (\text{CUSTOMER X AGENT}))$
- C. $\sigma_{\text{agent_cd} = \text{agent_code}} (\pi_{\text{cus_code, cus_lname, cus_zip, agent_code, agent_phone}} (\text{CUSTOMER X AGENT}))$
- D. All of the above

E. None of the above

UNION, INTERSECT, DIFFERENCE

STOREA

product_id	product_name
1	LG Nano91 75" 4K
2	TCL P725 65" 4K UHD
3	Sony X85J 75" Bravia

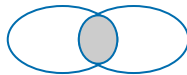


UNION (STOREA $\cup$ STOREB)

product_id	product_name
1	LG Nano91 75" 4K
2	TCL P725 65" 4K UHD
3	Sony X85J 75" Bravia
33	LG C1 48" Self Lit OLED 4K

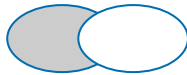
STOREB

product_id	product_name
1	LG Nano91 75" 4K
2	TCL P725 65" 4K UHD
33	LG C1 48" Self Lit OLED 4K



INTERSECT (STOREA $\cap$ STOREB)

product_id	product_name
1	LG Nano91 75" 4K
2	TCL P725 65" 4K UHD



DIFFERENCE (STOREA - STOREB)

product_id	product_name
3	Sony X85J 75" Bravia

Union compatible relations required

Q7. Relational Algebra

The following relations represent a karate dojo member training attendance:

SENSEI (sensei_id, sensei_name)

TRAINING_SCHEDULE (training_day, training_time, group_id, sensei_id)

ATTENDANCE (training_day, training_time, member_id, attendance_date)

MEMBER (member_id, member_name, member_dob, member_belt, group_id)

GROUP (group_id, group_name, group_age_range)

- A. Primary keys are underlined
- B. A karate member falls into one of the age level groups: Tiny Tiger (for 4-7 year old), Young Dragon (for 8-14 years old), or Adult (for 14+ years old) and owns a certain color of belt (e.g. white, green, brown or black)
- C. Sensei (Karate teachers) are scheduled to train an age level group of karate members in a particular day and time (e.g. Sensei Luke Nakamura trains Tiny Tiger members every Tuesday 5pm)
- D. A karate member may attend more than one training schedule of their age level group in a given week.

Write the relational algebra for the following query (**your answer must show an understanding of query efficiency**):

(2) Show the name, belt colour and attendance dates of the member with an id of 12345

ANSWER Q7

(2) Show the name, belt colour and attendance dates of the member with an id of 12345

$R2 = \pi \text{ member_name, member_belt, attendance_date } (\sigma \text{ member_id} = 12345 \text{ (MEMBER } \bowtie \text{ ATTENDANCE)})$

- this is the CANONICAL QUERY - not technically incorrect, but very inefficient, say member 12345 has only attended once in say 1000 tuples in ATTENDANCE. The join between MEMBER and ATTENDANCE yields, in such a scenario, 1000 tuples, 999 of which are unnecessary.

Your solution must demonstrate an understanding of efficiency:

$A2a = \pi \text{ member_id, attendance_date } (\sigma \text{ member_id} = 12345 \text{ ATTENDANCE})$

$A2b = \pi \text{ member_id, member_name, member_belt } (\sigma \text{ member_id} = 12345 \text{ MEMBER})$

$R2 = \pi \text{ member_name, member_belt, attendance_date } (A2a \bowtie A2b)$

Q8. Relational Algebra POST WORKSHOP TASK - answer available Sunday 5PM

The following relations represent a karate dojo member training attendance:

SENSEI (sensei_id, sensei_name)

TRAINING_SCHEDULE (training_day, training_time, group_id, sensei_id)

ATTENDANCE (training_day, training_time, member_id, attendance_date)

MEMBER (member_id, member_name, member_dob, member_belt, group_id)

GROUP (group_id, group_name, group_age_range)

- A. Primary keys are underlined
- B. A karate member falls into one of the age level groups: Tiny Tiger (for 4-7 year old), Young Dragon (for 8-14 years old), or Adult (for 14+ years old) and owns a certain color of belt (e.g. white, green, brown or black)
- C. Sensei (Karate teachers) are scheduled to train an age level group of karate members in a particular day and time (e.g. Sensei Luke Nakamura trains Tiny Tiger members every Tuesday 5pm)
- D. A karate member may attend more than one training schedule of their age level group in a given week.

Write the relational algebra for the following query (**your answer must show an understanding of query efficiency**):

- (3) Show the id, name and age level group name of members who were absent (did not attend any training) between 01-03-2021 and 31-03-2021 (inclusive).**

ANSWER Q8 - POST WORKSHOP TASK

(3) Show the id, name and age level group name of members who were absent (did not attend any training) between 01-03-2021 and 31-03-2021 (inclusive).

Required member details for all members:

A3a = π member_id, member_name, group_id (MEMBER)

All member id's who attended:

A3b = π member_id (σ attendance_date $\geq$ 01-03-2021 and
attendance_date $\leq$ 31-03-2021 (ATTENDANCE))

Required member details for those members who attended:

A3c = A3a $\bowtie$ A3b

All members MINUS members who attended (must be union compatible)

A3d = A3a - A3c

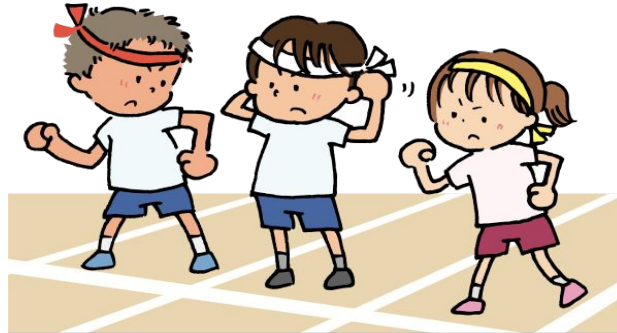
Join with group to get group_id

R = π member_id, member_name, group_name (A3d $\bowtie$ (π group_id, group_name (GROUP)))

Preparation for the workshop - ready, set

Please

- connect to Flux - flux.qa and be ready to answer questions
 - <https://flux.qa/BCUWQV>
- login to the Oracle database via **SQL Developer** (you will need to run the CISCO or Global VPN first if you are off campus)



Week 8

Update, Delete and Transaction Management

Workshop 2022 S1



MODIFYING ROWS USING UPDATE AND DELETE

UPDATE

- Changes the value of existing data.
- For example, it has been observed that when the TRAINING data for the drone system was added the description had an error and the hours for the course are incorrect.

```
UPDATE table  
SET column = (subquery) [, column = value, ...]  
[WHERE condition];
```

```
UPDATE training  
SET train_desc = 'DJI Hobby Drone Training',  
    train_hrs = 5  
WHERE train_code = 'DJIHY'
```

Update the drone
cost/hr by 10% for
all DJI Inspire 2
drones purchased
after the 31st
March 2021

```
UPDATE drone  
SET drone_cost_hr = drone_cost_hr * 1.1  
WHERE dt_code = (SELECT dt_code FROM drone_type WHERE  
                  dt_model = 'DJI Inspire 2')  
AND drone_pur_date > to_date('31-Mar-2021','dd-Mon-yyyy');
```

How do we find a string in the database?

- SQL search is case sensitive

Worksheet		Query Builder	
1	SELECT		
2	*		
3	FROM		
4	drone.manufacturer		
5	WHERE		
6	manuf_name = 'PARROT';		

Query Result x	
    SQL	All Rows Fetched: 0 in 0.011 seconds
 MANUF_ID	 MANUF_...

MANUF_ID	MANUF_NAME
10	DJI Da-Jiang Innovations
20	Parrot
30	SwellPro

Worksheet

Query Builder

1

2

3

4

5

6

7

SELECT

*

FROM

drone.manufacturer

WHERE

manuf_name = 'Parrot';

▶ Query Result ×

 SQL | All Rows Fetched: 1 in 0.01 seconds

	MANUF_ID	MANUF_NAME
1	20	Parrot

Since we cannot "know" the case of our data, SQL has two functions UPPER and LOWER used to modify case:

Worksheet

Query Builder

1

SELECT

2

*

3

FROM

4

drone.manufacturer

5

WHERE

6

LOWER(manuf_name) = 'parrot'

7

▶ Query Result

✕

🔍 🖨️ 🔄 🛑

SQL | All Rows Fetched: 1 in 0.012 seconds

MANUF_ID

MANUF_NAME

1

20 Parrot

Worksheet

Query Builder

```
1 SELECT
2 *
3 FROM
4   drone.manufacturer
5 WHERE
6   UPPER(manuf_name) = 'PARROT';
```

Query Result

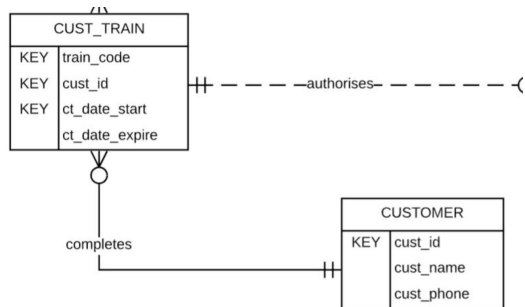
SQL | All Rows Fetched: 1 in 0.01 seconds

	MANUF_ID	MANUF_NAME
1	20	Parrot

DELETE

- Removing data from the database

```
DELETE FROM table  
[WHERE condition];
```



HiFlying Drones have a number of customers who have registered but never attended a training course, remove these customers from the CUSTOMER table

```
DELETE FROM customer  
WHERE cust_id NOT IN (SELECT DISTINCT cust_id FROM cust_train);
```

TRANSACTIONS

Transactions

- Consider the following situation.

When a drone returns from a rental two activities are required:

- *the return date is recorded, **and***
- *the drone flight time is updated to reflect the time the customer has flown*

Assume that rental number 239 was returned on the 27th April 2021 after having been flown for 120.3 minutes as part of this rental (the data is read from the drone). The SQL involved is:

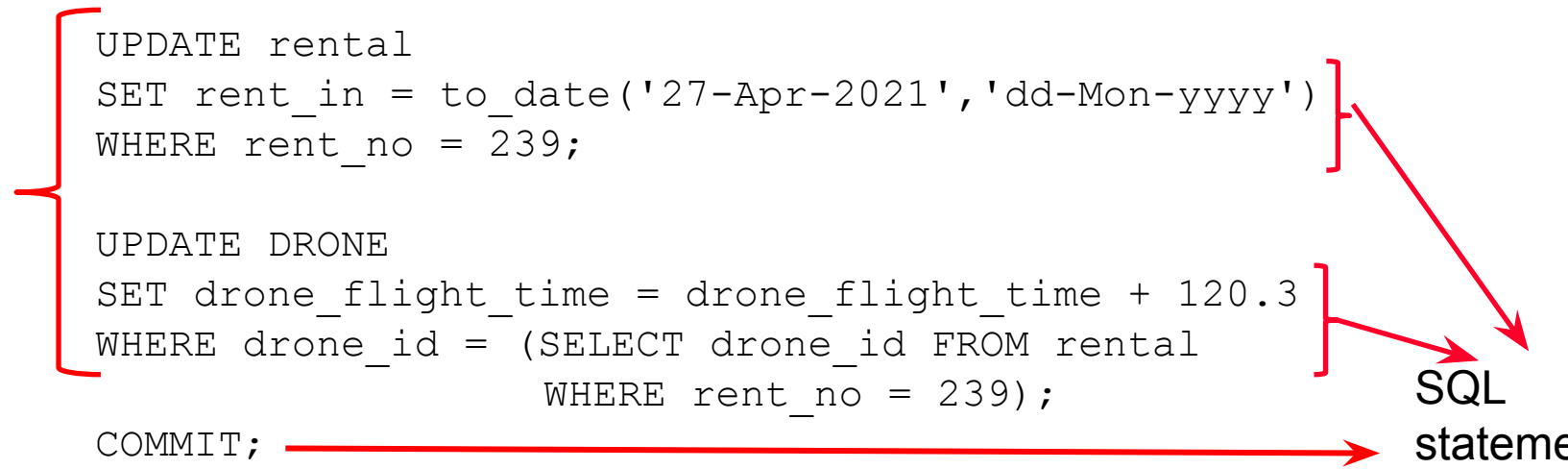
T
R
A
N
S
A
C
T
I
O
N

```
UPDATE rental
SET rent_in = to_date('27-Apr-2021','dd-Mon-yyyy')
WHERE rent_no = 239;

UPDATE DRONE
SET drone_flight_time = drone_flight_time + 120.3
WHERE drone_id = (SELECT drone_id FROM rental
                  WHERE rent_no = 239);

COMMIT;
```

SQL statements



All statements need to be run as a single logical unit operation.

Transaction Properties

- A transaction must have the following properties:
 - **Atomicity**
 - all database operations (SQL requests) of a transaction must be entirely completed or entirely aborted
 - **Consistency**
 - it must take the database from one consistent state to another
 - **Isolation**
 - it must not interfere with other concurrent transactions
 - data used during execution of a transaction cannot be used by a second transaction until the first one is completed
 - **Durability**
 - once completed the changes the transaction made to the data are durable, even in the event of system failure

Q1. Given the following transaction:

```
UPDATE rental
```

```
SET rent_in = to_date('27-Apr-2021','dd-Mon-yyyy')
```

```
WHERE rent_no = 239;
```

```
UPDATE DRONE
```

```
SET drone_flight_time = drone_flight_time + 120.3
```

```
WHERE drone_id = (SELECT drone_id FROM rental WHERE rent_no = 239);
```

```
COMMIT;
```

If the power for the database is lost after the first update to the RENTAL table, this (multiple answers possible):

- A. Can be ignored, no action is necessary
- B. Is a atomic property issue
- C. Is an isolation issue
- D. Is a consistency issue
- E. Is a durability issue

Consistency - Example

- Assume that the server lost its power after the execution of the first step of the drone return transaction. We now have a drone back but with the incorrect flight time - the database is inconsistent
- Consistency properties ensure, when the database is recovered, that RENTAL and DRONE tables will be returned to their states before the drone return process started.
- The last consistent state is *when the return step of the transaction has not been started*.

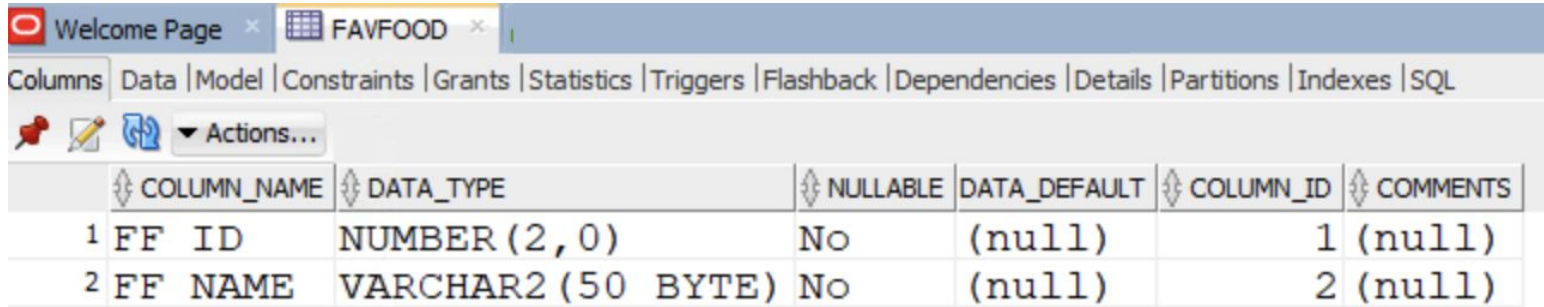
Durability - Example

- Assume the server lost power after the commit statement has been reached
 - Note that the change may not have been written to disk, it only existed in memory prior to the power loss
- The durability property ensures that the drones RENTAL rent_in date and DRONE drone_flight_time are maintained as the correct values when the server is restarted.

Transaction Management

- Follows the ACID properties.
- Transaction boundaries
 - Start
 - first SQL statement is executed (eg. Oracle)
 - Some systems have a BEGIN WORK type command
 - End
 - COMMIT or ROLLBACK
- Concurrency Management
- Restart and Recovery.

Insert into table WSHOP.FAVFOOD



The screenshot shows the Oracle SQL Developer interface with the 'FAVFOOD' table selected. The 'Columns' tab is active, displaying the table's structure. The table has two columns: 'ID' and 'NAME'. The 'ID' column is of type 'NUMBER(2,0)' and is not nullable. The 'NAME' column is of type 'VARCHAR2(50 BYTE)' and is not nullable. Both columns have a default value of '(null)' and no comments.

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	FF ID	NUMBER (2, 0)	No	(null)	1	(null)
2	FF NAME	VARCHAR2 (50 BYTE)	No	(null)	2	(null)

Please note your insert is being monitored and recorded, we will be displaying who entered what soon



Run the commands against the Oracle Database under the guidance of your workshop leader



Q2. Given two transactions:

T1 – R(X), W(X)

T2 – R(Y), W(Y), R(X), W (X)

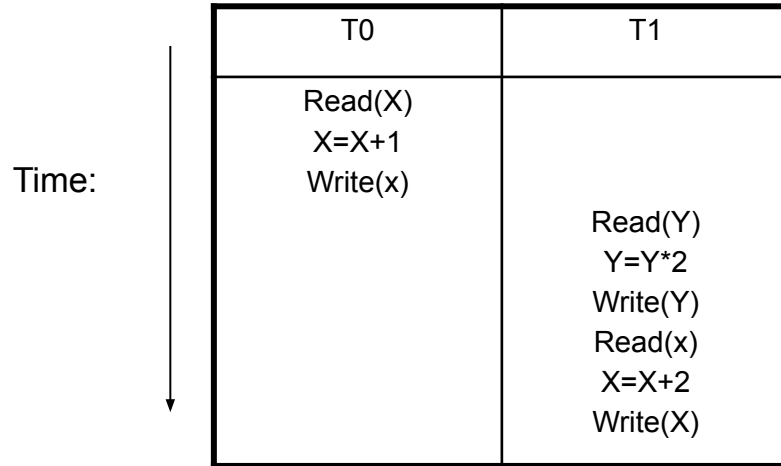
Where R(X) means Read(X) and W(X) means Write(X)

How many different sequences (schedules) are possible if serial execution is involved

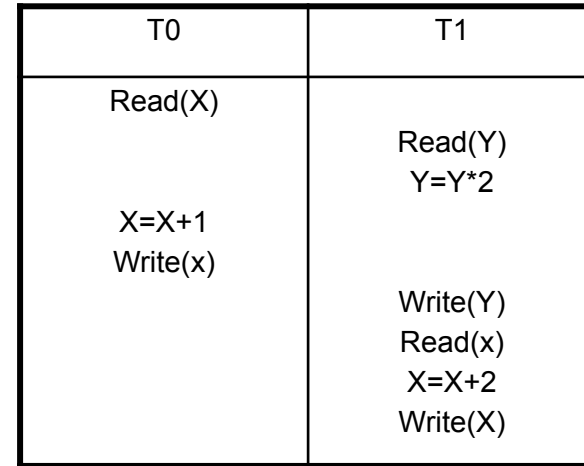
- A. 1
- B. It depends on the DBMS
- C. 6
- D. 2

Concurrency

Serial and *Interleaved* transactions.



Serial



Interleaved (non Serial)

Q3. Transaction T1 is calculating the total flight time of all drones while T2 is, at the same time, returning a drone and updating the drone_flight_time for a drone which has been returned. Calculating the total flight time involves read only access to the database.

This is an example of:

- A. Lost Update
- B. Uncommitted Data
- C. Inconsistent Retrieval
- D. None of these, this action causes no problems

The impact of interleaved transactions - Inconsistent Retrieval

TABLE 10.10

INCONSISTENT RETRIEVALS				
TIME	TRANSACTION	ACTION	VALUE	TOTAL
1	T1	Read PROD_QOH for PROD_CODE = '11QER/31'	8	8
2	T1	Read PROD_QOH for PROD_CODE = '13-Q2/P2'	32	40
3	T2	Read PROD_QOH for PROD_CODE = '1546-QQ2'	15	
4	T2	PROD_QOH = 15 + 10		
5	T2	Write PROD_QOH for PROD_CODE = '1546-QQ2'	25	
6	T1	Read PROD_QOH for PROD_CODE = '1546-QQ2'	25	(After) 65
7	T1	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	(Before) 88
8	T2	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	
9	T2	PROD_QOH = 23 - 10		
10	T2	Write PROD_QOH for PROD_CODE = '1558-QW1'	13	
11	T2	***** COMMIT *****		
12	T1	Read PROD_QOH for PROD_CODE = '2232-QTY'	8	96
13	T1	Read PROD_QOH for PROD_CODE = '2232-QWE'	6	102

Other possible issues:

- Lost Updates (update which is overwritten - see text)
- Uncommitted data (data read which is later rolled back - see text)

Concurrency Management - Solution

- Locking mechanism.
 - A mechanism to overcome the problems caused by interleaved transactions.
- A lock is an indicator that some part of the database is temporarily unavailable for update because:
 - one, or more, other transactions is reading it, or,
 - another transaction is updating it.
- A transaction must acquire a lock prior to accessing a data item and locks are released when a transaction is completed.
- Locking, and the release of locks, is controlled by a DBMS process called the Lock Manager.

Q4. A database using locking to support concurrency control will implement (multiple answers are possible):

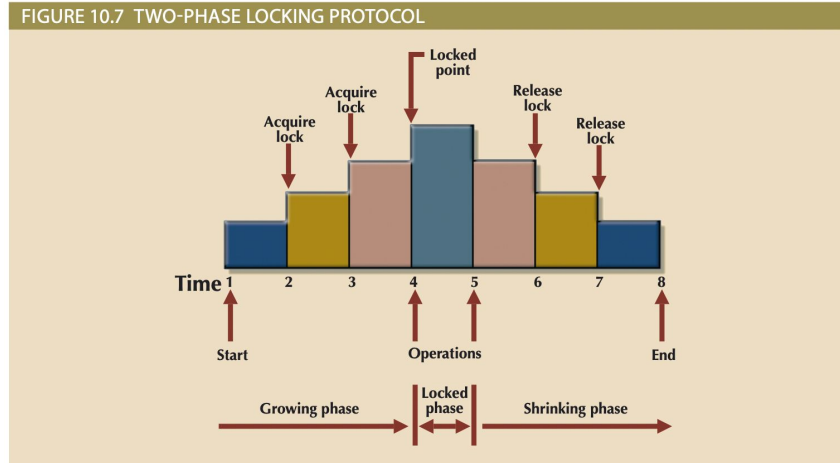
- A. Read locks
- B. Wait locks
- C. Timed locks
- D. Commit locks
- E. Write locks

Lock Types

- Shared lock. Multiple processes can simultaneously hold shared locks, to enable them to read without updating.
 - if a transaction T_i has obtained a shared lock (denoted by **S**) on data item **Q**, then T_i can **read** this item but not **write** to this item
- Exclusive lock. A process that needs to update a record must obtain an exclusive lock. Its application for a lock will not proceed until all current locks are released.
 - if a transaction T_i has obtained an exclusive lock (denoted **X**) on data item **Q**, then T_i can both **read** and **write** to item **Q**

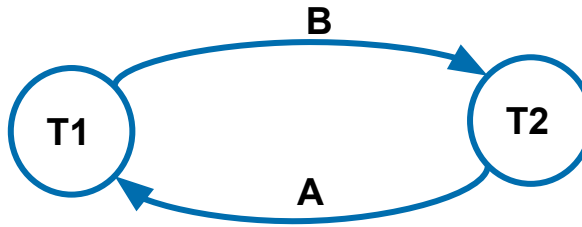
Two-Phase locking (2PL) to Ensure Serialisability

- Growing phase where locks are acquired
 - Once all locks acquired -> locked point
 - Transaction data changes made during locked point
- Shrinking phase where locks are released
 - Transaction cannot obtain a new lock during this phase



Lock - Problem

- Deadlock.



Scenario:

- Transaction 1 has an exclusive lock on data item A, and requests a lock on data item B.
- Transaction 2 has an exclusive lock on data item B, and requests a lock on data item A.

Result: Deadlock, also known as “deadly embrace”.

Each has locked a resource required by the other, and will not release that resource until it can either commit, or abort.

Unless some “referee” intervenes, neither will ever proceed.

Dealing with Deadlock

- Deadlock prevention
 - a transaction requesting a lock is aborted and restarted if it would cause a deadlock
- Deadlock avoidance
 - A transaction must acquire all the locks it requires before it updates any record.
 - If it cannot acquire a necessary lock, it releases all locks, and tries again later.
- Deadlock detection and recovery
 - Detection involves having the Lock Manager search the Wait-for tables for lock cycles.
 - Resolution involves having the Lock Manager force one of the transactions to abort, thus releasing all its locks.

Dealing with Deadlock

- If we discover that the system is in a state of deadlock, some of the transactions causing the deadlock must be aborted. Choosing which transaction to abort is called as *victim selection*.
- The algorithm for victim selection should generally avoid selecting transactions that have been running for a long time and that have performed many updates, and should try instead to select transactions that have not made any changes or that are involved in more than one deadlock cycle in the wait-for graph.

Database Restart and Recovery

- Restart
 - Soft crashes
 - loss of volatile storage, but no damage to disks. These necessitate restart facilities.
- Recovery
 - Hard crashes
 - hard crashes - anything that makes the disk permanently unreadable. These necessitate recovery facilities.
- Requires transaction log.

Transaction Log

- The **log**, or journal, tracks all transactions that update the database. It stores
 - For each transaction component (SQL statement)
 - Record for beginning of transaction
 - Type of operation being performed (update, delete, insert)
 - Names of objects affected by the transaction (the name of the table)
 - “Before” and “after” values for updated fields
 - Pointers to previous and next transaction log entries for the same transaction
 - The ending (COMMIT) of the transaction

The log should be written to a **multiple** separate physical devices from that holding the database, and must employ a force-write technique that ensures that every entry is immediately written to stable storage, that is, the log disk or tape.

Sample Transaction Log

TABLE 10.1

A TRANSACTION LOG

TRL_ID	TRX_NUM	PREV_PTR	NEXT_PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				



TRL_ID = Transaction log record ID

TRX_NUM = Transaction number

PTR = Pointer to a transaction log record ID

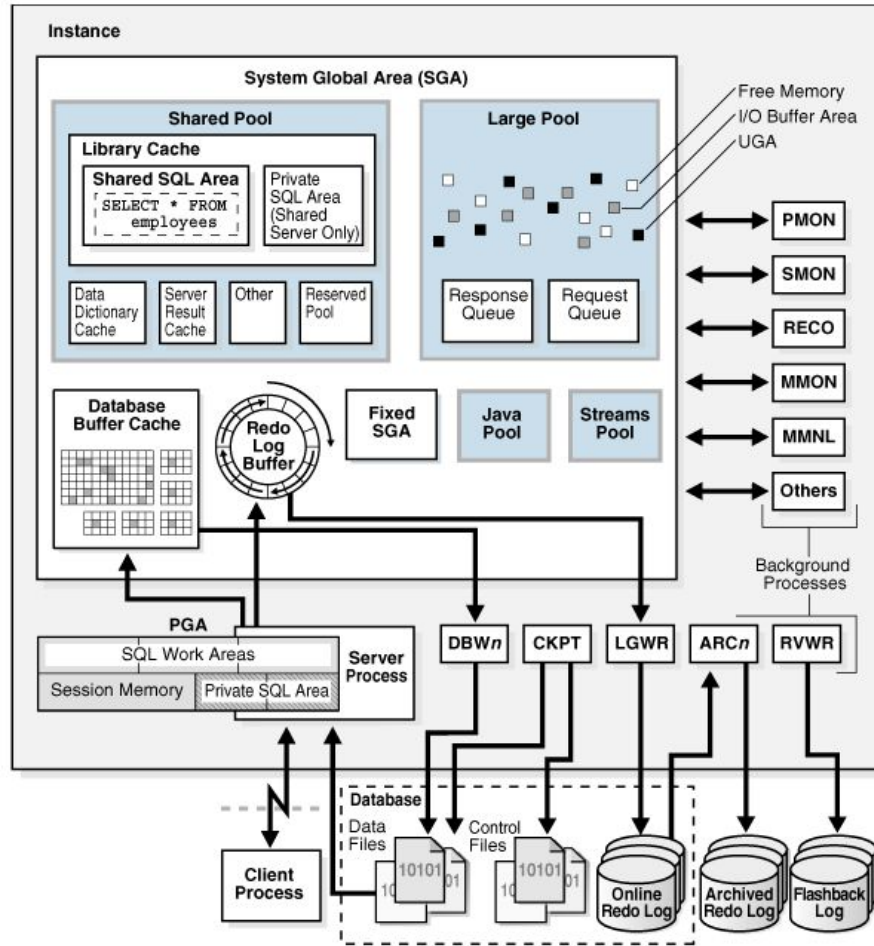
(Note: The transaction number is automatically assigned by the DBMS.)

Checkpointing

- Although there are a number of techniques for checkpointing, the following explains the *general* principle. A checkpoint is taken regularly, say every 15 minutes, or every 20 transactions.
- The procedure is as follows:
 - Accepting new transactions is temporarily halted, and current transactions are suspended.
 - Results of committed transactions are made permanent (force-written to the disk).
 - A checkpoint record is written in the log.
 - Execution of transactions is resumed.

Oracle database – *not examined*

INSTANCE (memory resident)



DATABASE
(on disk)

Write Through Policy

- The database is immediately updated by transaction operations during the transaction's execution, before the transaction reaches its commit point
- If a transaction aborts before it reaches its commit point a ROLLBACK or UNDO operation is required to restore the database to a consistent state
- The UNDO (ROLLBACK) operation uses the log before values

Restart Procedure for Write Through

Once the cause of the crash has been rectified, and the database is being restarted:

- **STEP 1: Using the log, compile REDO and UNDO lists**
 - The last checkpoint before the crash in the log file is identified. It is then read forward from, and two lists are constructed:
 - a REDO list containing the transaction-ids of transactions that were committed, and
 - a UNDO list containing the transaction-ids of transactions that never committed
- **STEP 2: UNDO incomplete or rolled back transactions** *starting from newest* (ROLLBACK using before images)
- **STEP 3: REDO committed transactions** *starting from oldest* (ROLLFORWARD using after images)

For each step the actions can be listed in the form:



— time →

with 1 ,2, 3 etc representing the transaction ids

DBMS recovery process using the *write through* method

Checkpoint



Time →

Transactions recorded in the log (green complete)

a	1	b	2	c	d	e	3	4	5	f	g	h	6	i	j	k	l	7	m
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Step 1: Using the log compile REDO and UNDO lists

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Step 2: UNDO incomplete or rolled back transactions starting from newest

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Step 3: REDO committed transactions starting from oldest

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

DBMS recovery process using the *write through* method

Checkpoint

Transactions recorded in the log (green complete)



a	1	b	2	c	d	e	3	4	5	f	g	h	6	i	j	k	l	7	m
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Step 1: Using the log compile REDO and UNDO lists

REDO list							UNDO list												
1	2	3	4	5	6	7	a	b	c	d	e	f	g	h	i	j	k	l	m

Step 2: UNDO incomplete or rolled back transactions starting from newest

m	l	k	j	i	h	g	f	e	d	c	b	a
---	---	---	---	---	---	---	---	---	---	---	---	---

Step 3: REDO committed transactions starting from oldest

1	2	3	4	5	6	7
---	---	---	---	---	---	---

An alternative - Deferred Write

- The database is updated only after the transaction reaches its commit point
- Required roll forward (committed transactions redone) but does not require rollback

DBMS recovery process using the *deferred write* method

Checkpoint

Transactions recorded in the log (green complete)



a	1	b	2	c	d	e	3	4	5	f	g	h	6	i	j	k	l	7	m
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

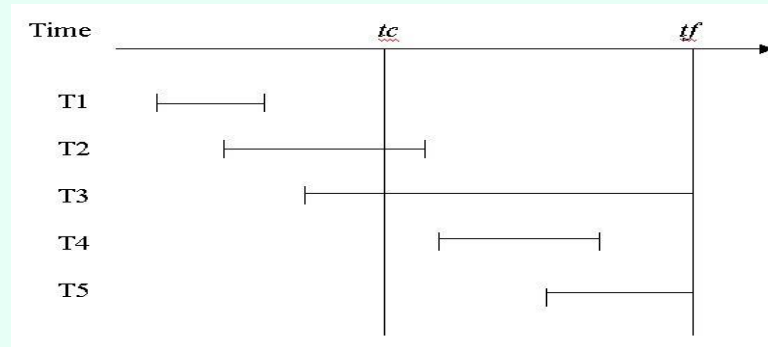
Step 1: Using the log compile REDO list

REDO list						
1	2	3	4	5	6	7

Step 2: REDO committed transactions starting from oldest

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Q5. For a write through system, which transaction/s will need to be **UNDONE (ROLLBACK)?**



tc = time of checkpoint
tf = time of failure

- A. T1
- B. T2
- C. T3
- D. T4
- E. T5
- F. None of them

Non Volatile Storage Recovery

- A hard crash involves physical damage to the disk, rendering it unreadable. This may occur in a number of ways:
 - Head-crash. The read/write head, which normally “flies” a few microns off the disk surface, for some reason actually contacts the disk surface, and damages it.
 - Accidental impact damage, vandalism or fire, all of which can cause the disk drive and disk to be damaged.
- After a hard crash, the disk unit, and disk must be replaced, reformatted, and then re-loaded with the database.

Backup

- A backup is a copy of the database stored on a different device to the database, and therefore less likely to be subjected to the same catastrophe that damages the database. (NOTE: A backup is not the same as a checkpoint.)
- Backups are taken say, at the end of each day's processing.
- Ideally, two copies of each backup are held, an on-site copy, and an off-site copy to cater for severe catastrophes, such as building destruction.
- Transaction log – backs up only the transaction log operations that are not reflected in a previous backup of the database.

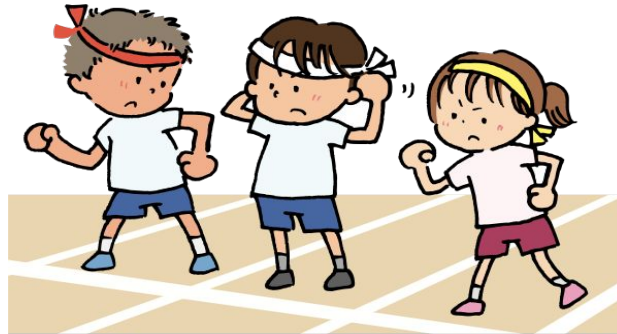
Recovery

- Rebuild the database from the most recent backup.
This will restore the database to the state it was in say, at close-of-business yesterday.
- **REDO** all committed transactions up to the time of the failure - no requirement for **UNDO**
- Known as ***Forward Recovery***

Preparation for the workshop - ready, set

Please

- connect to Flux - flux.qa and be ready to answer questions
 - <https://flux.qa/WZWLQG> (OnCampus) or <https://flux.qa/BCUWQV> (OnLine)
- login to Oracle:
 - SQL Developer (MoVE or Local) OR
 - ORDS <https://ora-fit.ocio.monash.edu:8440/ords/sql-developer>



Week 11

Non Relational Databases

Big Data

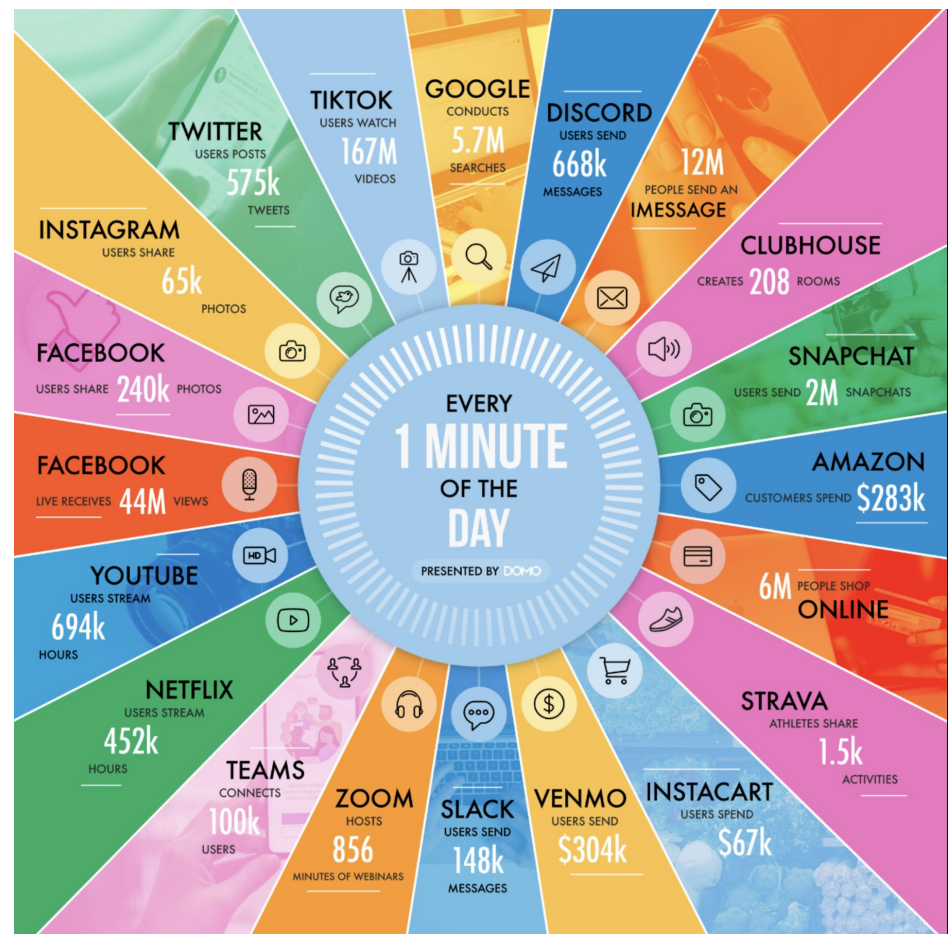
Workshop 2022 S1



Data Growth 2021

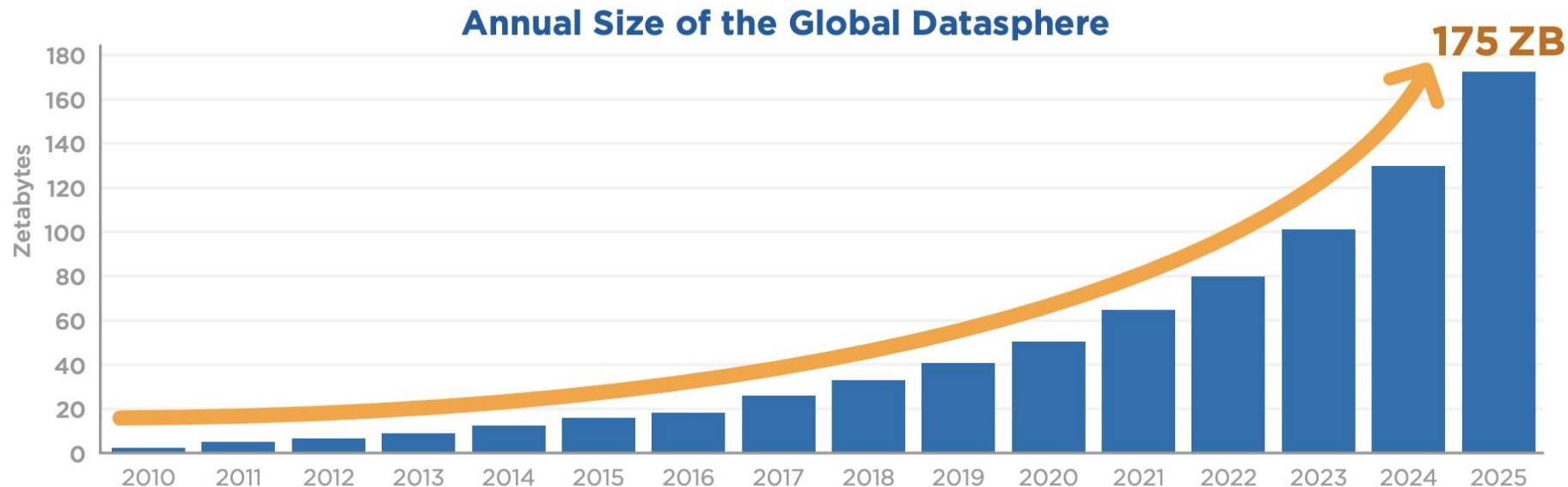
As of July 2021, the internet reaches 65% of the world's population and now represents 5.17 billion people—a 10% increase from January 2021. Of this total, 92.6 percent accessed the internet via mobile devices. According to Statista, the total amount of data consumed globally in 2021 was 79 zettabytes, an annual number projected to grow to over 180 zettabytes by 2025.

Global Internet Population Growth (IN BILLIONS)



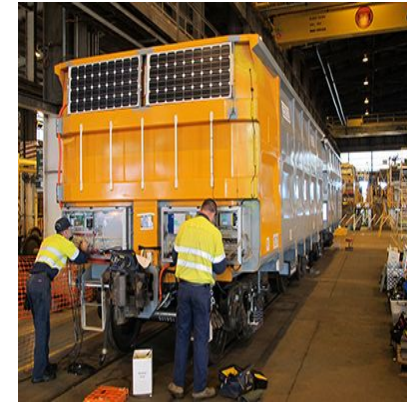
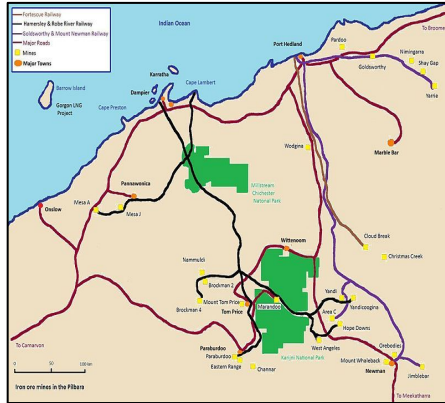
Data Growth

Figure 1 - Annual Size of the Global Datasphere



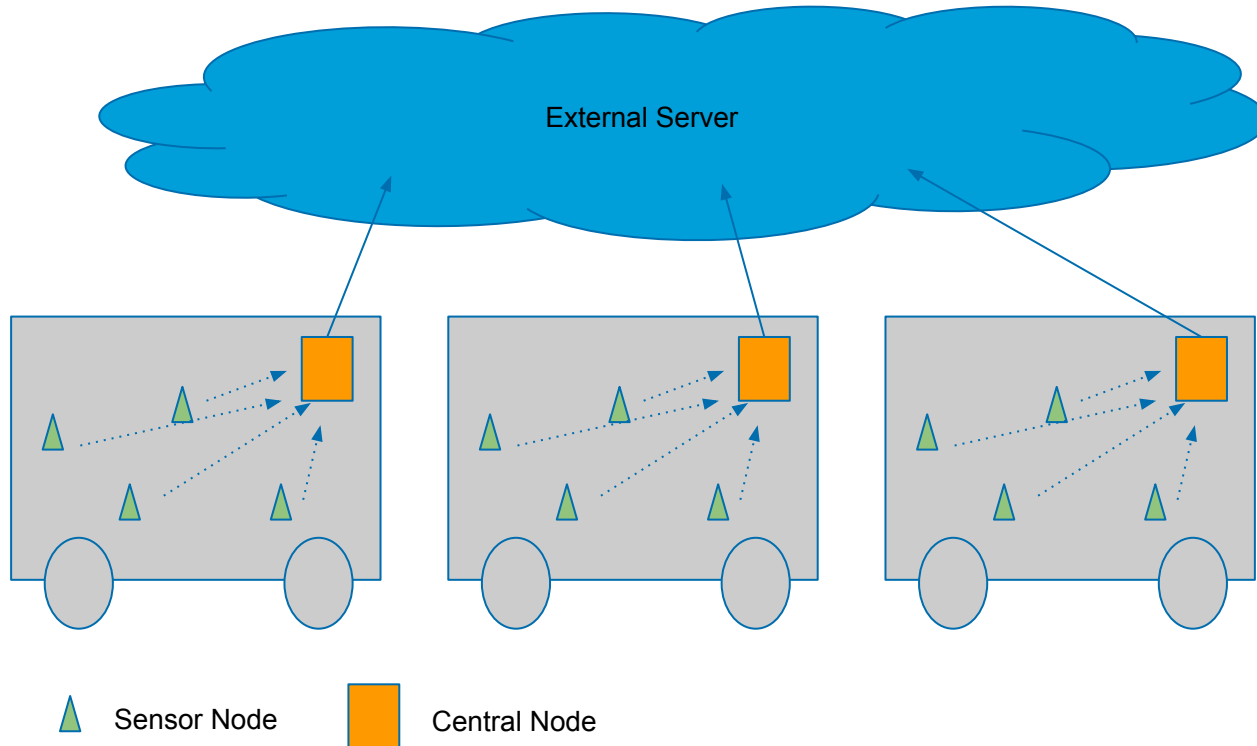
Source: Data Age 2025, sponsored by Seagate with data from IDC Global DataSphere, Nov 2018

Railway In Mining

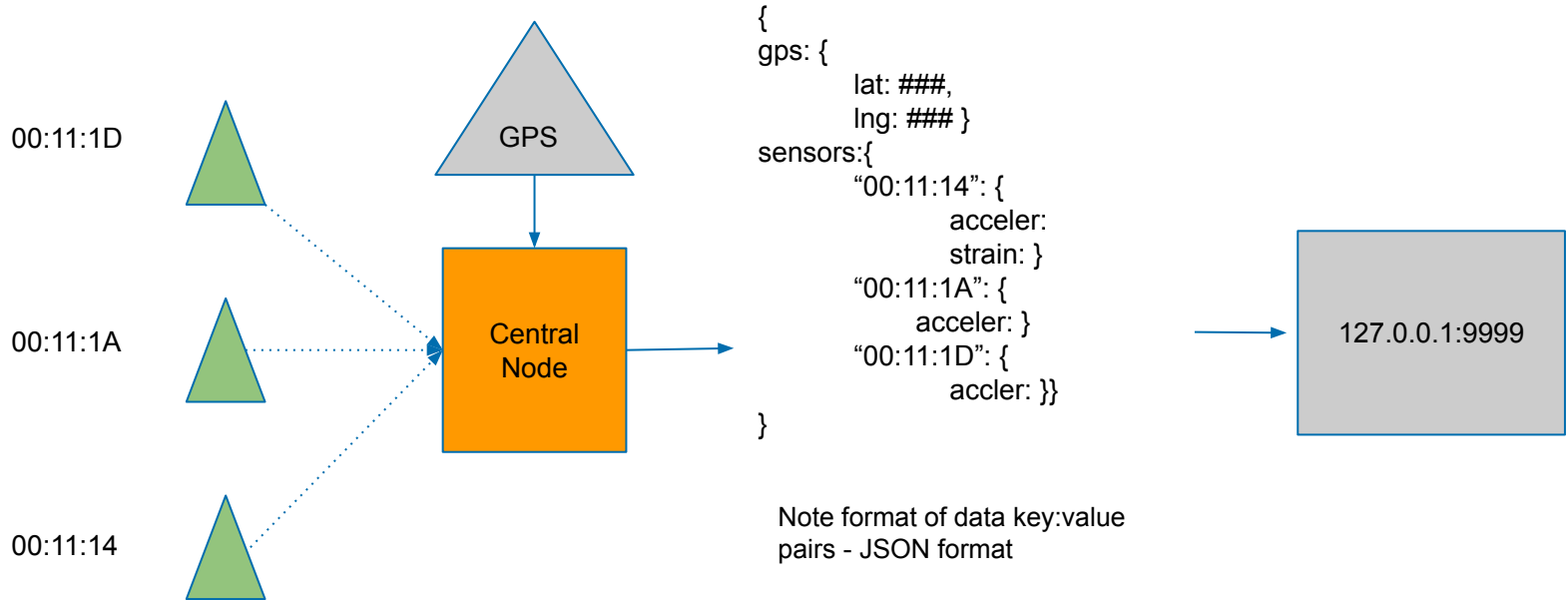


- Pilbara region, WA
- Trains perform round trips from the mining site to the port
- Loaded minerals and ores
- Length: > 2KM
- Load: > 10 Ton/car
- Speed: 5-10 Km/hr
- Instrumented Ore Car (IOC)
- Expensive Sensors
- Trained Professionals to maintain the sensors

Solution adopted: Network Structure



Central Node Process



How Big is the Data?

Quantity	Data Returned
Timestamp	12-Jun-2015; 09:35:15
Geo-location	N35°43.57518,W078°49.78314
Direction	ToPort
Acceleration	0.285g
Pressure	65psi
Ambient temperature	73 degrees F
Surface temperature	78 degrees F
Humidity	35%

➤ 16 Sensors

➤ 200 Ore Cars

➤ 25 Records Per Second

$16 * 200 * 25 = 80,000$ records/sec

Welcome to Ubuntu 14.04.3 LTS (GNU/Linux 3.13.0-46-generic x86_64)

* Documentation: <https://help.ubuntu.com/>

ubuntu@master:~\$ mongo

MongoDB shell version: 3.0.4

connecting to: test

2015-11-06T11:49:56.337+1100 I CONTROL [initandlisten]

2015-11-06T11:49:56.337+1100 I CONTROL [initandlisten] ** WARNING:

/sys/kernel/mm/transparent_hugepage/defrag is 'always'.

2015-11-06T11:49:56.337+1100 I CONTROL [initandlisten] ** We suggest setting it to 'never'

2015-11-06T11:49:56.337+1100 I CONTROL [initandlisten]

> Use IRT

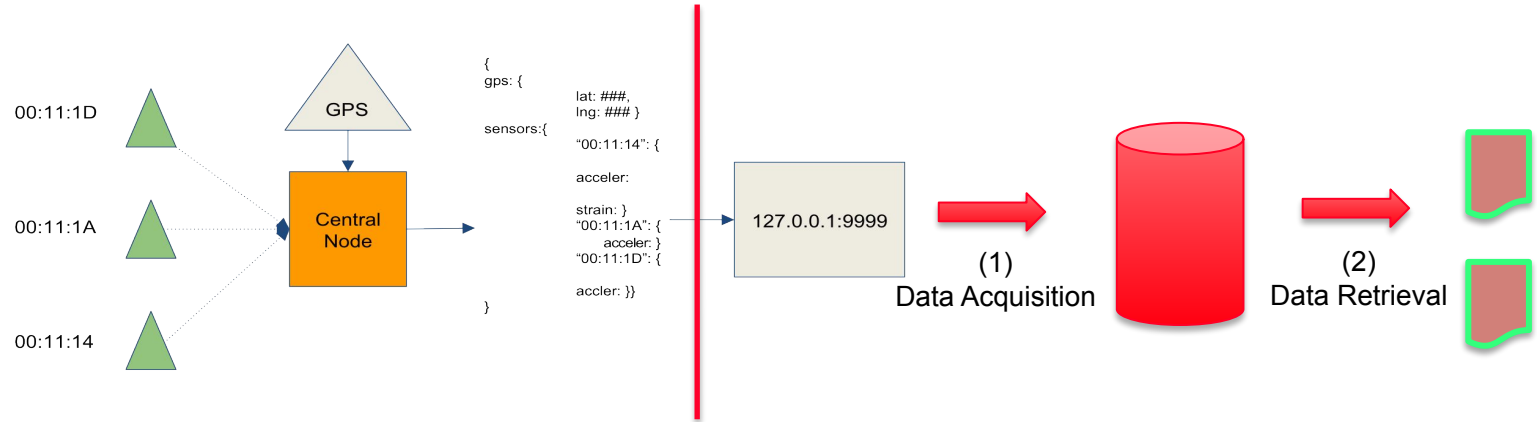
> db.sensordata.find().pretty()

```
{
  "_id" : ObjectId("5663ce2ce4b099b72ceca8c2"),
  "gps": { "GPSLat" : -21.63893238,"GPSLon" : 116.70659242},
  "SomatTime" : 74711,
  "CarOrient" : 30.2,
  "EorL" : 1,
  "Direction" : "ToPort ",
  "minSND" : 0,
  "iSegment" : 5876,
  "maxSND" : 0,
  "PipeA" : 0,
  "maxCFB" : 0,
  "minCFB" : 0,
  "Bounce" : 0,
  "minCFA" : 0,
  "maxCFA" : 0,
  "kmh" : 30.2,
  "PipeB" : 0,
  "Rock" : 0,
  "accR3" : 0,
  "accR4" : 0,
  "maxBounce" : 0,
  "LATACC" : 0
}
```

Type "it" for more

>

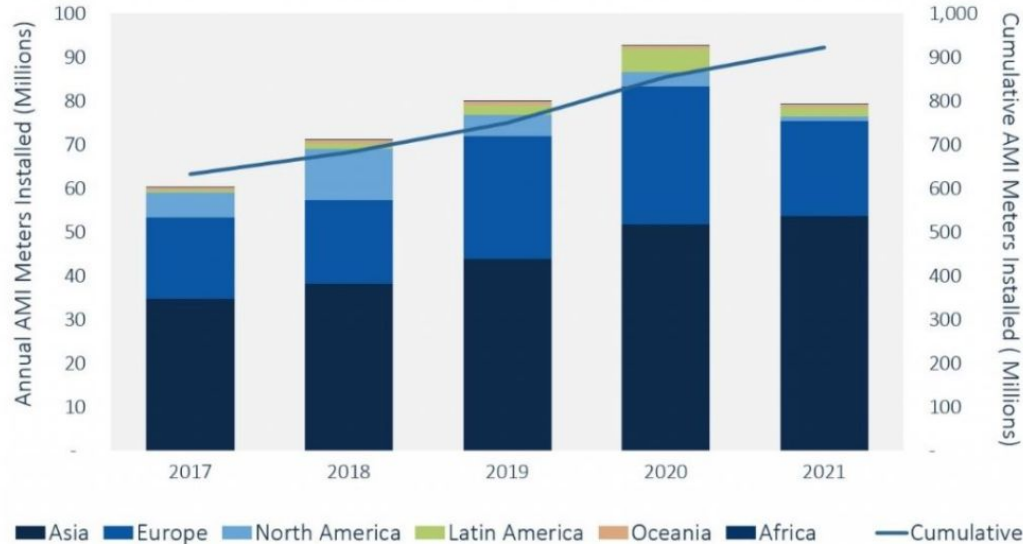
Big Data Processing



Two main problems:

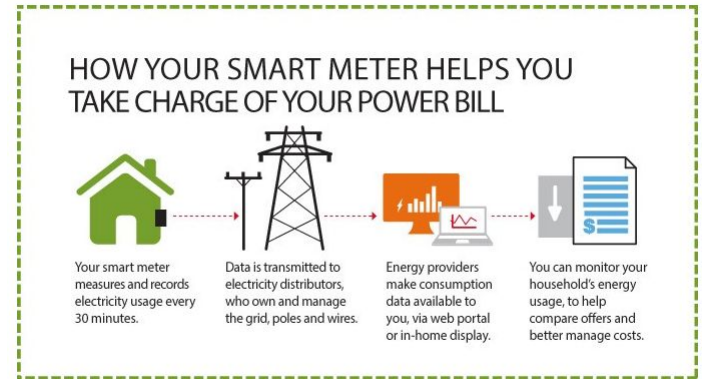
- (1) How to receive data ... massive amount of data
- (2) How to retrieve data ... very fast

Advanced metering infrastructure (AMI) - Smart Meters

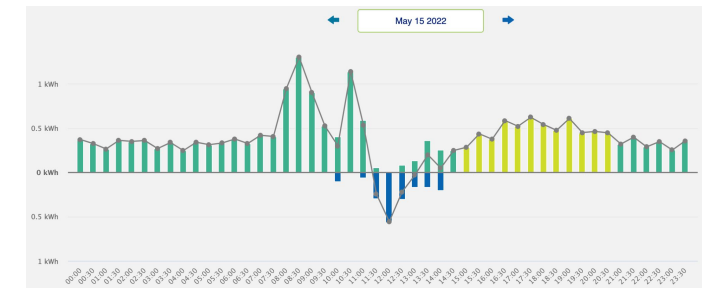


Source: GTM Research

<https://www.energynetworks.com.au/news/energy-insider/get-smart-when-will-australia-realise-the-benefits-of-smart-meters-and-iot/>



<https://www.victorianenergysaver.vic.gov.au/get-help-with-your-bills/smart-meters-and-how-they-work>



Q1. Which of the following is NOT a characteristic of Big Data (multiple selections are possible):

- A. Scaling Up
- B. Volume
- C. Veracity
- D. Variety
- E. Hadoop
- F. Velocity
- G. HDFS

Big Data Characteristics

- Volume
 - The quantity of data to be stored
- Velocity
 - The speed at which data enters the system and must be processed
- Variety
 - Variations in the structure of the data to be stored

FIGURE 14.1 ORIGINAL VIEW OF BIG DATA

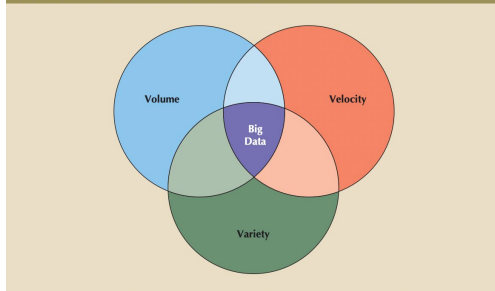
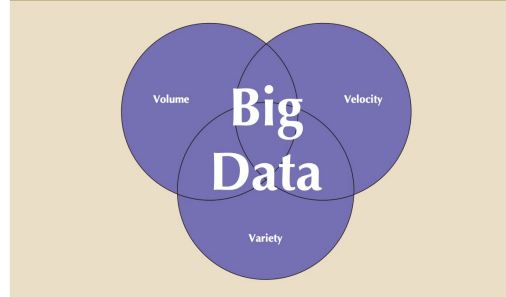


FIGURE 14.2 CURRENT VIEW OF BIG DATA



Big Data Characteristics: Volume

TABLE 14.1		
STORAGE CAPACITY UNITS		
TERM	CAPACITY	ABBREVIATION
Bit	0 or 1 value	b
Byte	8 bits	B
Kilobyte	1024* bytes	KB
Megabyte	1024 KB	MB
Gigabyte	1024 MB	GB
Terabyte	1024 GB	TB
Petabyte	1024 TB	PB
Exabyte	1024 PB	EB
Zettabyte	1024 EB	ZB
Yottabyte	1024 ZB	YB
* Note that because bits are binary in nature and are the basis on which all other storage values are based, all values for data storage units are defined in terms of powers of 2. For example, the prefix <i>kilo</i> typically means 1000; however, in data storage, a kilobyte = 2^{10} = 1024 bytes.		

- Scaling up: keeping the same number of systems but migrating each one to a larger system
- Scaling out: when the workload exceeds server capacity, it is spread out across a number of servers

Scaling continued

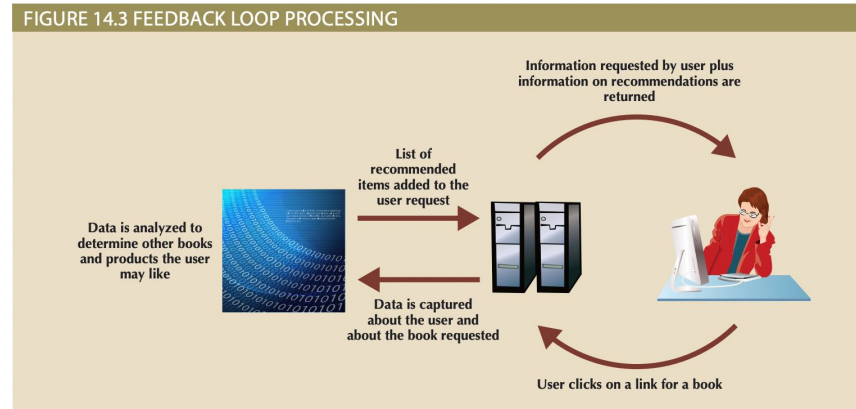
- Big players, notably Google and Amazon chose Scale Out
 - Lots and lots of smaller boxes ("commodity" servers)
 - Non relational structure
 - Google: Bigtable
 - <https://research.google/pubs/pub27898/>
 - <https://cloud.google.com/bigtable/docs/overview>
 - Used for wide range of apps Gmail, Google Earth, YouTube
 - Amazon: Dynamo
 - <http://www.read.seas.harvard.edu/~kohler/class/cs239-w08/decandia07dynamo.pdf>
 - Based on Dynamo: <https://aws.amazon.com/dynamodb/>

Scaling continued

- Term "NoSQL" coined by John Oskarsson in 2009 after calling a ..."free meetup about "open source, distributed, non relational databases" or NOSQL for short"...
 - <http://blog.oskarsson.nu/post/22996139456/nosql-meetup>
- Characteristics
 - Non relational,
 - mostly open source,
 - distributed (cluster friendly),
 - schema-less (no fixed storage schema)

Big Data Characteristics: Velocity

- Stream processing: focuses on input processing and requires analysis of data stream as it enters the system
 - CERN Large Hadron Collider 600TB per second ➡ 1GB per second
- Feedback loop processing: analysis of data to produce actionable results



Big Data Characteristics: Variety

- Structured data: fits into a predefined data model
 - Relational databases
 - Incoming data decomposed under normalisation rules to fit the data model
- Unstructured data: does not fit into a predefined model
 - Big Data requires that the data is captured in its natural format as generated without imposing a data model on it
- Semi structured data: combines elements of both

TABLE 14.2

ADDITIONAL Vs OF BIG DATA

CHARACTERISTIC	DESCRIPTION
Variability	Data meaning changes based on context.
Veracity	Data is correct.
Value (Viability)	Data can provide meaningful information.
Visualization	Data can be presented in such a way as to make it understandable.

Hadoop

- Hadoop is not a database
 - De facto standard for most Big Data storage and processing
 - Java-based framework for **distributing** and **processing** very large data sets across clusters of computers
 - <https://www.geeksforgeeks.org/hadoop-ecosystem/>
- Important components
 - Distribution
 - Hadoop Distributed File System (HDFS): low-level distributed file processing system that can be used directly for data storage
 - Processing
 - MapReduce: programming model that supports processing large data sets

Q2. The four main categories of NoSQL databases are (select multiple answers):

- A. Aggregate-aware
- B. Key-Value
- C. Document
- D. JSON
- E. Column-oriented
- F. Graph

NoSQL Data Models

- Key-value store

- Each item stored consists of a key and value pair (the value may be a numeric, a document, an image etc)

FIGURE 14.7 KEY-VALUE DATABASE STORAGE

Bucket = Customer

Key	Value
10010	"LName Ramas FName Alfred Initial A Areacode 615 Phone 844-2573 Balance 0"
10011	"LName Dunne FName Leona Initial K Areacode 713 Phone 894-1238 Balance 0"
10014	"LName Orlando FName Myron Areacode 615 Phone 222-1672 Balance 0"

- Oracle NoSQL database (community edition available)

- <https://www.oracle.com/au/database/technologies/related/nosql.html>

NoSQL Data Models continued

■ Document

- Each item is stored as a document (normally BSON or JSON document, but could be XML)
- Note the variable structure and embedded documents

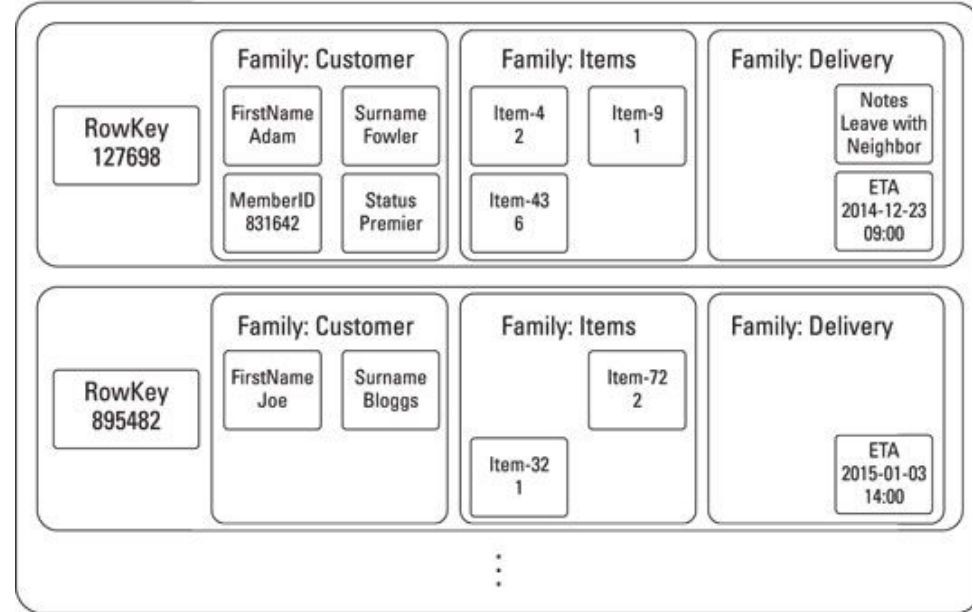
```
{  maker: "M.V. Agusta",  
  type: "sportsbike",  
  engine: {  
    type: "internal combustion",  
    cylinders: 4,  
    displacement: 750  
  },  
  rake:7,  
  trail:3.93  
}
```

```
{  maker : "M.V. Agusta",  
  type : "Helicopter",  
  engine : {  
    type : "turboshaft",  
    layout : "axial",  
    massflow : 1318  
  },  
  Blades : 4,  
  undercarriage : "fixed"  
}
```

MongoDB - <https://www.mongodb.com/>

NoSQL Data Models continued

- Column Family (also called Wide Column Store)
 - Key points to a set of multiple column values containing related data arranged by column family

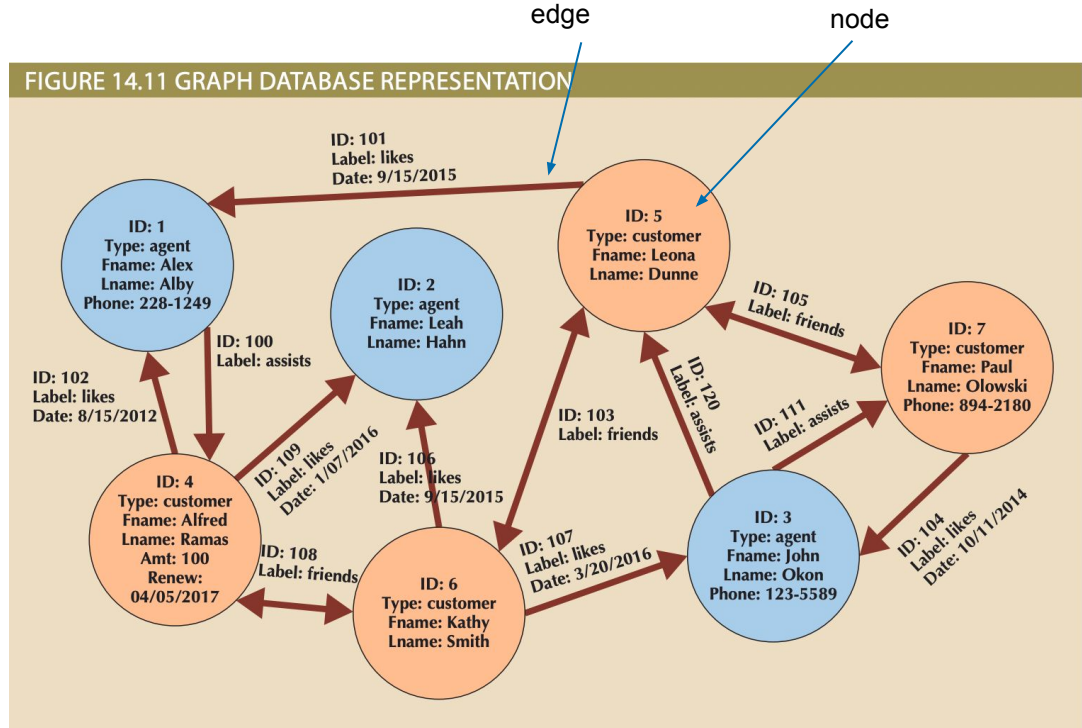


<https://www.dummies.com/programming/big-data/columnar-data-in-nosql/>

Cassandra (used on eBay): <https://cassandra.apache.org/>

NoSQL Data Models continued

- Graph - based on a graph structure
 - Unlike the previous three which are aggregation oriented, the graph model views data at a highly non aggregated level
 - Based on graph theory
 - Navigate via relationships (edges) between nodes
 - Examples
 - Neo4j
 - HyperGraphDB



<https://neo4j.com/docs/stable/cypher-cookbook-friend-finding.html>

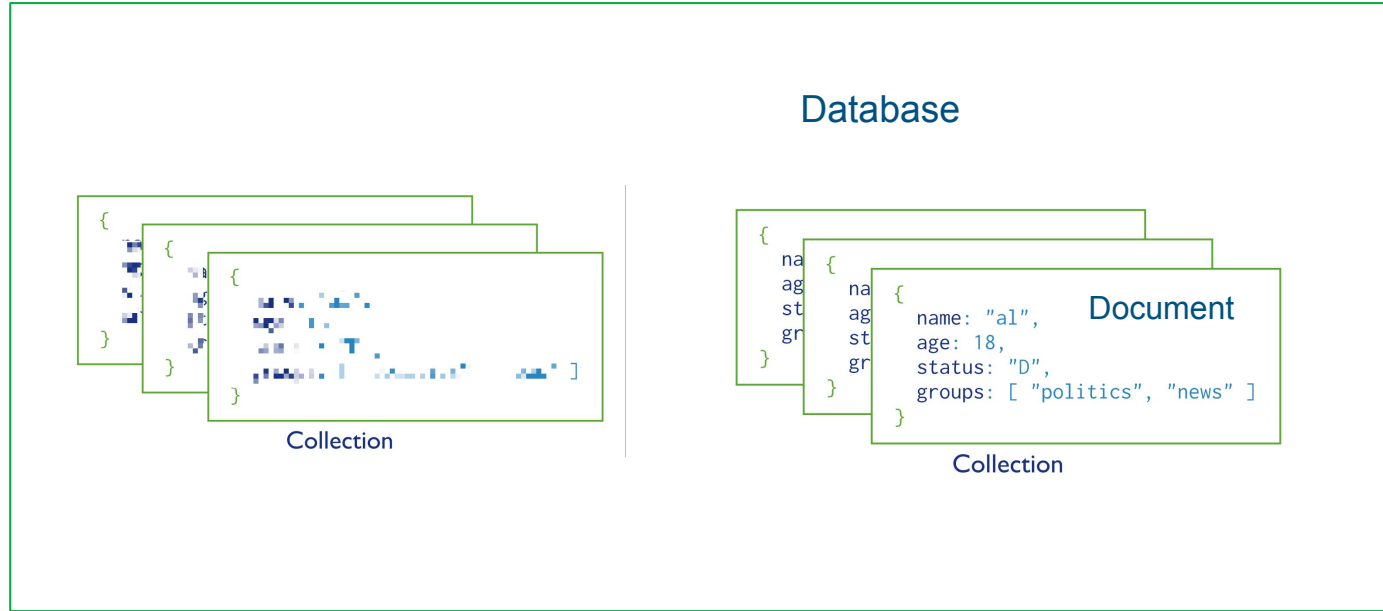
NoSQL Databases

- Comparison of NoSQL databases
 - <https://hostingdata.co.uk/nosql-database/> currently lists 200+ NoSQL databases, including some outside these four models.
- Data is distributed on multiple machines via:
 - **Sharding**
 - One copy of the data spread across multiple machines, or
 - **Replication**
 - Same data is spread across multiple machines, increased availability and resilience
 - **Mixtures of Sharding/Replication**
- Lots of interesting questions and research around consistency vs availability

MongoDB

- Document Database
 - Community edition available for download (not *required* for FIT9132 but may install - see Applied class)
 - <https://www.mongodb.com/download-center/community>
 - MongoDB Shell
 - <https://docs.mongodb.com/manual/tutorial/getting-started/>
 - Database
 - show dbs, use *dbname*, db.dropDatabase(), db (show current db)
 - Contains collections
 - show collections
 - » collection contains documents

MongoDB - Database



Documents -> Collections -> Database

Document structure

- MongoDB stores data records as BSON documents (binary JSON documents) - example from MongoDB manual:

```
{  
  name: "sue",  
  age: 26,  
  status: "A",  
  groups: [ "news", "sports" ]  
}
```

← field: value
← field: value
← field: value
← field: value

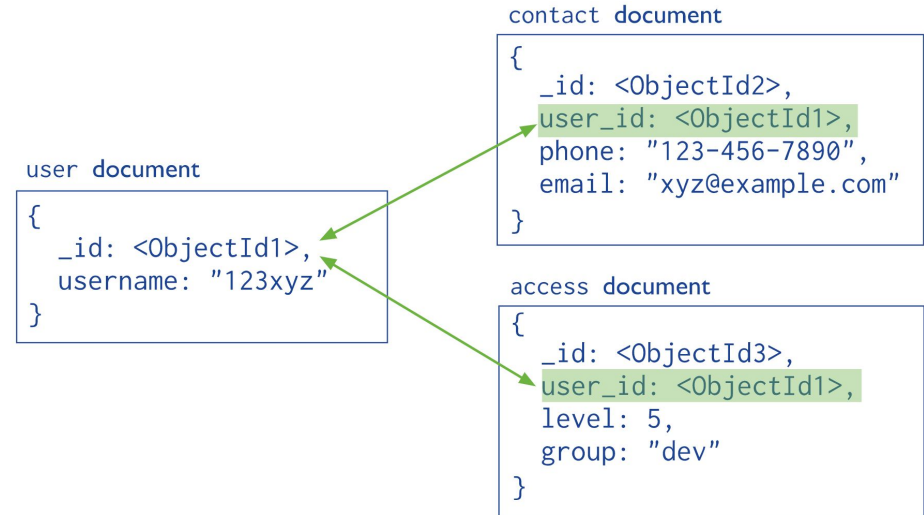
— Note that MongoDB manuals often do not quote various things such as name above - *they should be quoted*.

- Document composed of field-values pairs
 - Field names may be enclosed in quotes (allows spaces in name)
 - "groups" field above holds an array of strings, marked with []

Relationships – structuring documents



Denormalised – Embedded Documents



Normalised – References

Document sample - Drone Rentals

```
{
  "drone_id": 111,
  "type": {
    "code": "PAPR",
    "model": "Parrot Pro",
    "manufacturer": "Parrot"
  },
  "carrying_capacity": 5,
  "pur_date": "2020-03-20",
  "pur_price": 4200,
  "total_flighttime": 100,
  "cost_per_hour": 45,
  "RentalInfo": [
    {
      "rent_no": 11,
      "bond": 150,
      "rent_out": "2020-04-26",
      "rent_in": "2020-04-28",
      "custtrain_id": 10
    }
  ]
}
```

- type - sub document
- RentalInfo - array of sub documents

Generate JSON object from Oracle Select - JSON functions

set pagesize 50 -- this sets the output page size, prevents the output heading appearing every 5 lines

SELECT

```
JSON_OBJECT(  
  'drone_id' VALUE drone_id,  
  'type' VALUE JSON_OBJECT (  
    'code' VALUE dt_code,  
    'model' VALUE dt_model,  
    'manufacturer' VALUE manuf_name  
  ),  
  'carrying_capacity' VALUE dt_carry_kg,  
  'pur_date' VALUE to_char(drone_pur_date, 'YYYY-MM-DD'),  
  'pur_price' VALUE drone_pur_price,  
  'total_flighttime' VALUE drone_flight_time,  
  'cost_per_hour' VALUE drone_cost_hr,  
  'RentalInfo' VALUE JSON_ARRAYAGG (  
    JSON_OBJECT (  
      'rent_no' VALUE rent_no,  
      'bond' VALUE rent_bond,  
      'rent_out' VALUE to_char(rent_out, 'YYYY-MM-DD'),  
      'rent_in' VALUE to_char(rent_in, 'YYYY-MM-DD'),  
      'custtrain_id' VALUE ct_id  
    )  
    ORDER BY rent_no  
  ) FORMAT JSON )  
|| ','
```

FROM ...

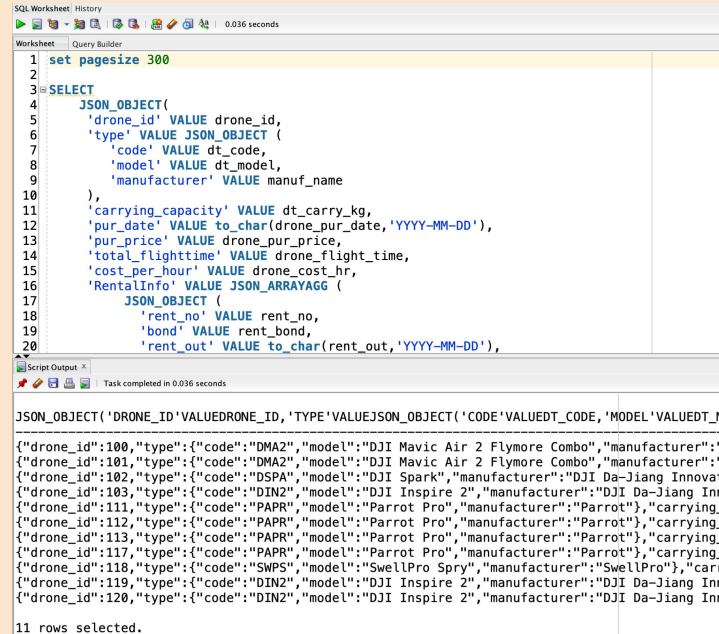
GROUP BY ...

ORDER BY

drone_id;

<https://docs.oracle.com/en/database/oracle/oracle-database/12.2/adjsn/generation.html>

Collect DRONE data from Oracle



```
SQL Worksheet: History
0.036 seconds

Worksheet: Query Builder

1 set pagesize 300
2
3 SELECT
4   JSON_OBJECT(
5     'drone_id' VALUE drone_id,
6     'type' VALUE JSON_OBJECT (
7       'code' VALUE dt_code,
8       'model' VALUE dt_model,
9       'manufacturer' VALUE manu_name
10    ),
11     'carrying_capacity' VALUE dt_carry_kg,
12     'pur_date' VALUE to_char(drone_pur_date, 'YYYY-MM-DD'),
13     'pur_price' VALUE drone_pur_price,
14     'total_flighttime' VALUE drone_flight_time,
15     'cost_per_hour' VALUE drone_cost_hr,
16     'RentalInfo' VALUE JSON_ARRAYAGG (
17       JSON_OBJECT (
18         'rent_no' VALUE rent_no,
19         'bond' VALUE rent_bond,
20         'rent_out' VALUE to_char(rent_out, 'YYYY-MM-DD'),
```

```
JSON_OBJECT('DRONE_ID' VALUE DRONE_ID, 'TYPE' VALUE JSON_OBJECT('CODE' VALUE DT_CODE, 'MODEL' VALUE DT_M
{
  "drone_id": 100, "type": {
    "code": "DMA2", "model": "DJI Mavic Air 2 Flymore Combo", "manufacturer": "DJI"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 101, "type": {
    "code": "DMA2", "model": "DJI Mavic Air 2 Flymore Combo", "manufacturer": "DJI"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 102, "type": {
    "code": "DSPA", "model": "DJI Spark", "manufacturer": "DJI"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 103, "type": {
    "code": "DIN2", "model": "DJI Inspire 2", "manufacturer": "DJI"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 111, "type": {
    "code": "PAPR", "model": "Parrot Pro", "manufacturer": "Parrot"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 112, "type": {
    "code": "PAPR", "model": "Parrot Pro", "manufacturer": "Parrot"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 113, "type": {
    "code": "PAPR", "model": "Parrot Pro", "manufacturer": "Parrot"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 117, "type": {
    "code": "PAPR", "model": "Parrot Pro", "manufacturer": "Parrot"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 118, "type": {
    "code": "SWPS", "model": "SwellPro Spry", "manufacturer": "SwellPro"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 119, "type": {
    "code": "DIN2", "model": "DJI Inspire 2", "manufacturer": "DJI"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}, {
  "drone_id": 120, "type": {
    "code": "DIN2", "model": "DJI Inspire 2", "manufacturer": "DJI"
  }, "carrying_capacity": 1000, "pur_date": "2019-01-01", "pur_price": 1000, "total_flighttime": 1000, "cost_per_hour": 1000, "RentalInfo": [
    {
      "rent_no": 1, "bond": 1000, "rent_out": "2019-01-01"
    }
  ]
}
]
11 rows selected.
```

1. Create a text document dronedata.txt using the output via [Visual Studio Code for the Web](#)
2. Format as JSON and save as dronedata.json

mongoDB - CRUD: CREATE

- create collection by inserting documents
 - `db.collection.insertOne ( ..... JSON ..... );`
 - `db.collection.insertMany` - insert an array of JSON documents
 - `insertMany ([ JSON1, JSON2, ...]);`

```
drone> db.drone.rent.insertOne({
...   "drone_id": 100,
...   "type": {
...     "code": "DMA2",
...     "model": "DJI Mavic Air 2 Flymore Combo",
...     "manufacturer": "DJI Da-Jiang Innovations"
...   },
...   "carrying_capacity": 0,
...   "pur_date": "2020-01-13",
...   "pur_price": 1494,
...   "total_flighttime": 100,
...   "cost_per_hour": 15,
...   "RentalInfo": [
...     {
...       "rent_no": 1,
...       "bond": 100,
...       "rent_out": "2020-02-20",
...       "rent_in": "2020-02-20",
...       "custtrain_id": 1
...     },
...     {
...       "rent_no": 4,
...       "bond": 100,
...       "rent_out": "2020-02-22",
...       "rent_in": "2020-02-25",
...       "custtrain_id": 4
...     }
...   ]
... });
{
  acknowledged: true,
  insertedId: ObjectId("6281eeefc6d5875c821f89cb")
}
```

Add the first document to MongoDB

<https://www.mongodb.com/docs/v4.4/tutorial/getting-started/>



The screenshot shows the MongoDB Web Shell interface. On the left, a terminal window displays the following text: "MongoDB Web Shell", "Click to connect", "Connecting...", "MongoDB shell version v4.4.6", "connecting to: mongodb://127.0.0.1:27017/?authSource=admin&compressors=disabled&gssapiServiceName=mongodb", "Implicit session: session { 'id' : UUID('38477088-a500-473d-b3e3-68937a69af18') }", "MongoDB server version: 4.4.6", "type 'help' for help", and ">>>". On the right side of the terminal window, there are three circular buttons: "Full" (grey), "Reset" (orange), and "Clear" (grey).

Which version of MongoDB is running: **db.version()**

Which databases do you have access to: **show dbs**

Create/use the drone database: **use drone**

Which database am I in: **db**

What collections do I have in this database: **show collections**

Add the first document to MongoDB continued

```
type help for help
>>> use drone
switched to db drone
>>> db.droneinsert.insertOne({
...   "drone_id": 100,
...   "type": {
...     "code": "DMA2",
...     "model": "DJI Mavic Air 2 Flymore Combo",
...     "manufacturer": "DJI Da-Jiang Innovations"
...   },
...   "carrying_capacity": 0,
...   "pur_date": "2020-01-13",
...   "pur_price": 1494,
...   "total_flighttime": 100,
...   "cost_per_hour": 15,
...   "RentalInfo": [
...     {
...       "rent_no": 1,
...       "rent_date": "2020-01-13"
...     }
...   ]
... })
```

Full

Reset

Clear

Success:

```
{
  "acknowledged" : true,
  "insertedId" : ObjectId("6281b9d67c28b58e72528d61")
}
```

Now insert the remainder in one insertMany (note the use of an array [] to contain the set of documents)

mongoDB - CRUD: RETRIEVE

- Documents retrieved by find method on collection
 - `db.dronerent.find({});` or `db.dronerent.find({}).pretty()` (older versions)

- find all

```
[drone> show collections
dronerent
[drone> db.dronerent.find()
[
  {
    _id: ObjectId("6281dee6c6d5875c821f89b6"),
    drone_id: 100,
    type: {
      code: 'DMA2',
      model: 'DJI Mavic Air 2 Flymore Combo',
      manufacturer: 'DJI Da-Jiang Innovations'
    },
    carrying_capacity: 0,
    pur_date: '2020-01-13',
    pur_price: 1494,
    total_flighttime: 100,
    cost_per_hour: 15,
    RentalInfo: [
      {
        rent_no: 1,
        bond: 100,
        rent_out: '2020-02-20',
        rent_in: '2020-02-20',
        custtrain_id: 1
      },
      {
        rent_no: 4,
        bond: 100,
        rent_out: '2020-02-22',
        rent_in: '2020-02-25',
        custtrain_id: 4
      }
    ]
  }
]
```

mongoDB - CRUD: RETRIEVE continued

- Limit output to specified field (project fields) 1 display 0 suppress

```
[drone> db.dronerent.find({}, {"_id": 0})
[
  {
    drone_id: 100,
    type: {
      code: 'DMA2',
      model: 'DJI Mavic Air 2 Flymore Combo',
      manufacturer: 'DJI Da-Jiang Innovations'
    },
    carrying_capacity: 0,
    pur_date: '2020-01-13',
    pur_price: 1494,
    total_flighttime: 100,
    cost_per_hour: 15,
    RentalInfo: [
      {
        rent_no: 1,
        bond: 100,
        rent_out: '2020-02-20',
        rent_in: '2020-02-20',
        custtrain_id: 1
      },
      {
        rent_no: 4,
        bond: 100,
        rent_out: '2020-02-22',
        rent_in: '2020-02-25',
        custtrain_id: 4
      }
    ]
  },
  {
    drone_id: 101,
    type: {
      code: 'DMA2',
      model: 'DJI Mavic Air 2 Flymore Combo',
      manufacturer: 'DJI Da-Jiang Innovations'
    }
  }
]
```

```
drone> db.dronerent.find({}, {"_id": 0, "drone_id": 1, "type.model": 1})
[
  { drone_id: 100, type: { model: 'DJI Mavic Air 2 Flymore Combo' } },
  { drone_id: 101, type: { model: 'DJI Mavic Air 2 Flymore Combo' } },
  { drone_id: 102, type: { model: 'DJI Spark' } },
  { drone_id: 103, type: { model: 'DJI Inspire 2' } },
  { drone_id: 111, type: { model: 'Parrot Pro' } },
  { drone_id: 112, type: { model: 'Parrot Pro' } },
  { drone_id: 113, type: { model: 'Parrot Pro' } },
  { drone_id: 117, type: { model: 'Parrot Pro' } },
  { drone_id: 118, type: { model: 'SwellPro Spry' } },
  { drone_id: 119, type: { model: 'DJI Inspire 2' } },
  { drone_id: 120, type: { model: 'DJI Inspire 2' } }
]
```

```
[drone> db.dronerent.find({}).count()
11
drone> █
```

- count documents returned

mongoDB - CRUD: RETRIEVE continued

- Find some documents
 - Predicate Operators: <https://docs.mongodb.com/manual/reference/operator/query/>
 - Example: `{ <field>: { $eq: <value> } } => db.inventory.find( { qty: { $eq: 20 } } )`

Comparison

For comparison of different BSON type values, see the [specified BSON comparison order](#).

Name	Description
<code>\$eq</code>	Matches values that are equal to a specified value.
<code>\$gt</code>	Matches values that are greater than a specified value.
<code>\$gte</code>	Matches values that are greater than or equal to a specified value.
<code>\$in</code>	Matches any of the values specified in an array.
<code>\$lt</code>	Matches values that are less than a specified value.
<code>\$lte</code>	Matches values that are less than or equal to a specified value.
<code>\$ne</code>	Matches all values that are not equal to a specified value.
<code>\$nin</code>	Matches none of the values specified in an array.

Logical

Name	Description
<code>\$and</code>	Joins query clauses with a logical AND returns all documents that match the conditions of both clauses.
<code>\$not</code>	Inverts the effect of a query expression and returns documents that do <i>not</i> match the query expression.
<code>\$nor</code>	Joins query clauses with a logical NOR returns all documents that fail to match both clauses.
<code>\$or</code>	Joins query clauses with a logical OR returns all documents that match the conditions of either clause.

Remember these should be quoted eg. "\$eq"

mongoDB - CRUD: RETRIEVE continued

```
db.dronerent.find ({})  
db.dronerent.find ({}).count()  
db.dronerent.find ({},{ "_id": 0, "drone_id": 1, "type.model": 1})
```

Find some documents

- a. find the details of drone id 102
- b. find the details of all drones of type DIN2
- c. find the details of all drones which have a carrying capacity > 4
 - display drone id, model and cost per hour
- d. find the details of all drones which have a carrying capacity <= 5 and a cost per hour of < 50
 - display drone id, carrying capacity and cost per hour
- e. how many drones are still on loan?
- f. which drones are still out on loan
 - display drone id, when the drone went out, check your answer by doing a count first

mongoDB - CRUD: RETRIEVE continued

Find some documents

- a. find the details of drone id 102

```
db.dronerent.find ({"drone_id": {"$eq": 102}})
```

- b. find the details of all drones of type DIN2

```
db.dronerent.find ({"type.code": {"$eq": "DIN2"}})
```

- c. find the details of all drones which have a carrying capacity > 4
 - display drone id, model and cost per hour

```
db.dronerent.find (  
  {"carrying_capacity": {"$gt": 4}},  
  {"drone_id": 1, "type.model": 1, "cost_per_hour": 1, "_id": 0}  
)
```

mongoDB - CRUD: RETRIEVE continued

- d. find the details of all drones which have a carrying capacity ≤ 5 and a cost per hour of < 50
- display drone id, carrying capacity and cost per hour

```
db.dronerent.find (
  { "$and": [{"carrying_capacity": {"$lte": 5}}, {"cost_per_hour": {"$lt": 50}}]},
  {"drone_id": 1, "carrying_capacity": 1, "cost_per_hour": 1, "_id": 0}
)
```

- e. how many drones are still on loan?

```
db.dronerent.find ({"RentalInfo.rent_in": {"$eq": null}}).count()
```

- f. which drones are still out on loan
- display drone id, when the drone went out, check your answer by doing a count first

```
db.dronerent.find (
  {"RentalInfo.rent_in": {"$eq": null}},
  {"drone_id":1, "RentalInfo.rent_out":1, "_id":0 }
)
```

mongoDB - CRUD: UPDATE

- Update documents via update or updateOne
 - uses \$set to assign value
 - updateOne ({*query condition*},{*update to carry out*})

```
drone> db.droneRent.find (
...   {"drone_id": {"$eq": 103}},
...   {"type.model": 1, "pur_date": 1, "total_flighttime": 1, "_id": 0 }
... )
[
  {
    type: { model: 'DJI Inspire 2' },
    pur_date: '2020-01-13',
    total_flighttime: 200
  }
]
drone> db.droneRent.updateOne (
...   { "drone_id": {"$eq": 103}}, {"$set": {"total_flighttime": 230} }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
drone> db.droneRent.find( { "drone_id": { "$eq": 103 } }, { "type.model": 1, "pur_date": 1, "total_flighttime": 1, "_id": 0 })
[
  {
    type: { model: 'DJI Inspire 2' },
    pur_date: '2020-01-13',
    total_flighttime: 230
  }
]
drone>
```

mongoDB - CRUD: UPDATE

- Update within an array
 - **\$** placeholder to update the first element that matches the query condition

```
drone> db.droneRent.find (
...   { "$and": [ { "drone_id": { "$eq": 118 } }, { "RentalInfo.rent_out": "2021-04-13" } ] },
...   { "RentalInfo": { "$slice": -1 }, "_id": 0 }
... )
[
  {
    drone_id: 118,
    type: { code: 'SWPS', model: 'SwellPro Spry', manufacturer: 'SwellPro' },
    carrying_capacity: 0,
    pur_date: '2020-04-01',
    pur_price: 1599,
    total_flighttime: 56.3,
    cost_per_hour: 16,
    RentalInfo: [
      {
        rent_no: 25,
        bond: 180,
        rent_out: '2021-04-13',
        rent_in: null,
        custtrain_id: 18
      }
    ]
  }
]
drone> db.droneRent.updateOne (
...   { "$and": [ { "drone_id": { "$eq": 118 } }, { "RentalInfo.rent_out": "2021-04-13" } ] },
...   { "$set": { "RentalInfo.$.rent_in": "2021-04-18" } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
drone>
```

mongoDB - CRUD: UPDATE Result

- Update within an array

BEFORE

```
drone> db.dronerent.find (
...   { "$and": [ { "drone_id": { "$eq": 118 } }, { "RentalInfo.rent_out": "2021-04-13" } ] },
...   { "RentalInfo": { "$slice": -1 }, "_id": 0 }
... )
[... ]
[
  {
    drone_id: 118,
    type: { code: 'SWPS', model: 'SwellPro Spry', manufacturer: 'SwellPro' },
    carrying_capacity: 0,
    pur_date: '2020-04-01',
    pur_price: 1599,
    total_flighttime: 56.3,
    cost_per_hour: 16,
    RentalInfo: [
      {
        rent_no: 25,
        bond: 180,
        rent_out: '2021-04-13',
        rent_in: null,
        custtrain_id: 18
      }
    ]
  }
]
```

AFTER

```
drone> db.dronerent.find (
...   { "$and": [ { "drone_id": { "$eq": 118 } }, { "RentalInfo.rent_out": "2021-04-13" } ] },
...   { "RentalInfo": { "$slice": -1 }, "_id": 0 }
... )
[... ]
[
  {
    drone_id: 118,
    type: { code: 'SWPS', model: 'SwellPro Spry', manufacturer: 'SwellPro' },
    carrying_capacity: 0,
    pur_date: '2020-04-01',
    pur_price: 1599,
    total_flighttime: 56.3,
    cost_per_hour: 16,
    RentalInfo: [
      {
        rent_no: 25,
        bond: 180,
        rent_out: '2021-04-13',
        rent_in: '2021-04-18', ←
        custtrain_id: 18
      }
    ]
  }
]
drone> █
```

mongoDB - CRUD: DELETE

- Delete a document
 - via `db.dronerent.deleteOne` or `db.dronerent.deleteMany`

```
[drone> db.dronerent.find({}).count()
11
[drone> db.dronerent.deleteOne({"drone_id": {"$eq": 103}})
{ acknowledged: true, deletedCount: 1 }
[drone> db.dronerent.find({}).count()
10
[drone> db.dronerent.deleteMany({"carrying_capacity": {"$gt": 4}})
{ acknowledged: true, deletedCount: 6 }
[drone> db.dronerent.find({}).count()
4
drone> █
```

- Remove current database (local client only)
 - `db.dropDatabase()`

2022 Final Assessment

- **2 HOUR 10 minutes**

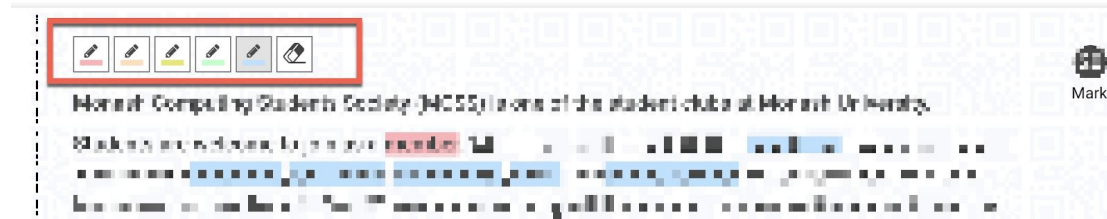
- Time includes reading time
- **eExam, Closed Book with permitted sites/software, online supervision, no aids (eg. dictionary)**
 - **Full access to the Unit Moodle teaching material, unit software and databases**
 - Throughout the exam the screen of your device will be recorded (supervisor watching, AI supervision and post exam analysis)
 - Must not access multiple devices or screens (may only use one screen)
 - Access to any site or software which has not been approved will result in exam being cancelled
 - Posting of questions to other sites/users will/has resulted in suspension/expulsion from course (Chegg and such sites are monitored)

2022 Final Assessment continued

- Paper will be available via the eExam platform
 - <https://www.monash.edu/exams/electronic-exams>
 - **Note Academic Staff for this unit have NO control over this platform, all issues/concerns MUST be raised with the eExam staff**
 - [Getting help and support - Current students \(monash.edu\)](#)
 - This link should be used for both pre exam, during exam and post exam issues
 - "Chrome in Incognito mode must be used for **all** eExams" - from [eExam site](#)
 - ensure Chrome will open text files and PDF files within your browser, not download
 - Complete the e-exam [General Knowledge Practice](#) to see form of eExam papers

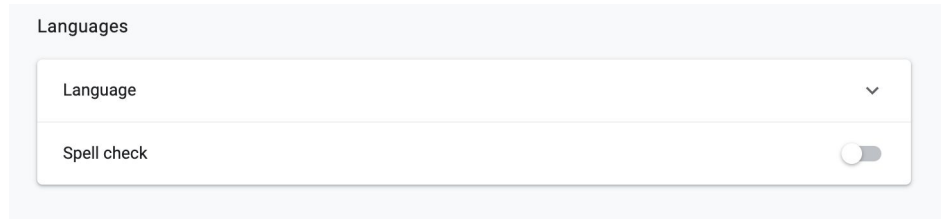
2022 Final Assessment continued

- During the exam you can highlight text as you read the question for emphasis



- Spelling check is turned off in the exam paper, however you also need to turn it off in your browser

For Chrome:



2022 Final Assessment Structure

- 100 marks 40% of your final mark for this unit.
 - Minimum to pass - overall:
 - 45% in-semester, 45% final assessment and 50% overall
- Questions:
 - Cover theory and application
 - Timing - 100 marks, 120 mins - **1 mark/minute target**
 - SQL/NoSQL case study will be released on Moodle the day before the scheduled exam, **do not bring** the case study to the Final Assessment (no hard copy material) or **discuss it in the Ed forums**
 - Oracle tables for the case study will be available during the exam time **ONLY - tables must not be created in your account**
- Final Assessment when - **your responsibility to determine**

Permitted Resources during Exam

▪ Online Resources

- FIT9132 2022 Semester 1 Moodle site:
 - <https://lms.monash.edu/course/view.php?id=136238>
 - *Sample solutions available directly as files not in zip archive*
- Oracle ORDS Server:
 - <https://ora-fit.ocio.monash.edu:8440/ords/sql-developer>
- MoVE (File Explorer and SQL Developer only):
 - <https://move.monash.edu/>
- Visual Studio Code for the Web
 - <https://vscode.dev/>
- MongoDB Shell:
 - <https://www.mongodb.com/docs/manual/tutorial/getting-started/>
- No collaboration tools or websites other than those listed above are permitted

Permitted Resources during Exam continued

- **Locally installed software**
 - Oracle SQL Developer
 - MongoDB Shell
- Copy and Paste is permitted **ONLY**
 - where your answer is developed in the Oracle or MongoDB environment, and
 - where indicated permitted for the particular question on the exam paper
- Copy and Paste must always be your own content, and not the duplication of anyone else's work

Exam cover sheet - Instructions

Instructions

- This is an **Closed Book with Specifically Permitted Items** exam.
 - **ONLY** the following online resources may be used:
 - FIT9132 2022 Semester 1 Moodle site:
<https://lms.monash.edu/course/view.php?id=136238>
 - Oracle ORDS Server:
<https://ora-fit.ocio.monash.edu:8440/ords/sql-developer>
 - MoVE (File Explorer and SQL Developer only):
<https://move.monash.edu/>
 - Visual Studio Code for the Web
<https://vscode.dev/>
 - MongoDB Shell:
<https://www.mongodb.com/docs/manual/tutorial/getting-started/>
 - **ONLY** the following locally installed software may be used
 - Oracle SQL Developer
 - MongoDB Shell
 - No collaboration tools or websites other than those listed above are permitted
 - Your answer for Q3 - Q8 (Section B and Section C) should be developed in the appropriate database environment. Copy and Paste is permitted **ONLY** for these answers, but must always be your own content, and not the duplication of anyone else's work.
- Please answer **ALL** questions.

Authorised Materials



Closed Book with Specifically permitted items



No Calculator

Websites: FIT9132 Sem 1 Moodle Site; Oracle ORDS Server; MoVE; VS Code for Web; MongoDB Shell
Software: Oracle SQL Developer; MongoDB Shell.

Rules

During an exam, you must not have in your possession any item/material that has not been authorised for your exam. This includes books, notes, paper, electronic device/s, mobile phone, smart watch/device, calculator, pencil case, or writing on any part of your body. Any authorised items are listed above. Items/materials on your desk, chair, in your clothing or otherwise on your person will be deemed to be in your possession.

You must not retain, copy, memorise or note down any exam content for personal use or to share with any other person by any means following your exam.

You must comply with any instructions given to you by an exam supervisor.

As a student, and under Monash University's Student Academic Integrity procedure, you must undertake your in-semester tasks, and end-of-semester tasks, including exams, with honesty and integrity. In exams, you must not allow

2022 Final Assessment

- All content specified during your semester of study in this unit is examinable either directly or may be required in the context of a question. This includes but is not limited to:
 - Pre-reading (weekly Coronel & Morris chapters), HP5 review exercises
 - Workshop Slides and Videos
 - Applied Session Notes/Solutions, and
 - all other Moodle Materials (except where explicitly stated NOT EXAMINABLE).

2022 Mock Final Assessment

- Serves to provide an overview of the general structure of the final assessment paper only.
- Available Friday 27th May at 5 PM
 - Sample solution available Wednesday, 8th June 9AM (please attempt the mock paper before accessing the sample solution)
 - Both sample paper and solution close 6 hrs before Scheduled Final Assessment (Exam)
 - Forums also close 6 hrs before Scheduled Final Assessment (Exam)
- Link on Moodle under - "Scheduled Final Assessment" on Assessments page
- To protect the integrity of the paper: NO ACTUAL FINAL ASSESSMENT PAPER QUESTIONS are included; and the COMPOSITION OF THE SUB QUESTIONS are SUBJECT TO CHANGE.
- Suggest you leave it until you can attempt it under time limit of 2 hours 10 minutes (**time control is critical**)
- Allows multiple attempts

Week 2 & 5 Data Modelling

Not directly examined

- Conceptual vs Logical Level
- Entity
 - Strong vs weak
 - Associative entity
- Types of attributes
- Relationship
 - Connectivity - type : one-to-one, one-to-many, many-to-many
 - Cardinality
 - Participation
 - Identifying vs Non-identifying.
- Mapping from Conceptual to Logical
 - E.g. Mapping many-to-many

Week 3 – Relational Model **EXAMINED**

- Relational model properties.
- Keys
 - Superkey, Candidate Key, Primary Key
 - Foreign Key
- Data Integrity
 - Entity integrity
 - Referential Integrity
- Relational Algebra
 - Understanding of efficiency

Week 4 – Normalisation

Not directly examined

- UNF to 3NF
 - **Mapping form to UNF**
 - UNF to 1NF – remove repeating group.
 - 1NF to 2NF – remove partial dependency (**general** definition)
 - 2NF to 3NF – remove transitive dependency.
- Dependency diagrams
 - For example: cust_id → cust_name, cust_phone, cust_balance
- Be careful in choosing the PK!
- Mapping a set of 3NF relations to a logical model

- CREATE TABLE statements
 - Primary key definition
 - Foreign key definition
 - Other Constraints
- ALTER
- INSERT
 - Adherence to referential integrity constraints
 - Order of insertion
- Oracle Sequence
- UPDATE (DML)
- DELETE (DML)

- Single table retrieval with predicate
- Join
 - Natural join
 - Outer join
- Aggregate functions
- Set Operators
- Subquery
- Oracle functions
 - TO_CHAR, TO_DATE, NVL, UPPER, LOWER, ROUND, RTRIM/LTRIM, LPAD/ RPAD
- PAYROLL SQL revision available on Moodle

Week 8 – Transaction Management **EXAMINED**

- Transactions
 - transaction boundaries (start and end)
 - use of commit/rollback
- ACID properties.
- Transaction problems.
- Transaction management with locks.
- Wait For Graphs
- Two-Phase Locking
- Restart and Recovery using Transaction Log.

Week 11 – noSQL EXAMINED

- Characteristics of Big Data
- Four major noSQL models
- Role of Hadoop (basic only)
- Using SQL to generate JSON document
- MongoDB CRUD commands
 - C: insert (One and Many)
 - R: find (predicates, count(), pretty())
 - U: update (One and Many)
 - D: delete (One and Many)

Week 12 – Database Future Directions

- The content of week 12's lecture
 - Database Trends
 - Future directions

Is NOT examinable (questions related to this week's new content will not appear on the paper)